GOBIERNO DEL
ESTADO DE MÉXICO

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

UNIDAD DE ESTUDIOS SUPERIORES
SAN JOSÉ DEL RINCÓN

Plataforma Integral para la Gestión de Medicamentos y Generación de Citas del Centro de Orientación Familiar y Salud, I.A.P. (Hospital Mazahua)

PROYECTO DE RESIDENCIA

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

PRESENTA:
RODOLFO CRUZ IBARRA

ASESOR:
I.S.C. ANDREA VAZQUEZ TELLEZ

octubre de 2026.

GOBIERNO DEL
ESTADO DE MÉXICO

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

UNIDAD DE ESTUDIOS SUPERIORES
SAN JOSÉ DEL RINCÓN

Plataforma Integral para la Gestión de Medicamentos y Generación de Citas del Centro de Orientación Familiar y Salud, I.A.P. (Hospital Mazahua)

PROYECTO DE RESIDENCIA

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

PRESENTA:
RODOLFO CRUZ IBARRA

ASESOR:
I.S.C. ANDREA VAZQUEZ TELLEZ

octubre de 2026.

ÍNDICE DEL CONTENIDO

INTRODUCCIÓN 9
PLANTEAMIENTO DEL PROBLEMA 10
JUSTIFICACIÓN 11
OBJETIVOS 12
General: 12
Específicos: 12
CAPÍTULO I. GENERALIDADES DE LA INSTITUCIÓN 13
1.1 Antecedentes 14
1.2 Ubicación. 16
1.3 Misión 16
1. 4 Misión: 16
1.5 Valores: 16
1.6 Logotipos de la institución. 17
1.7 Organigrama general. 18
CAPÍTULO II. MARCO TEÓRICO 19
2.1 Figma 20
2.1.1 Sistemas de diseño y traspaso a desarrollo: 20
2.2 Visual Studio Code 20
2.2.1 Protocolo de lenguaje (LSP) y personalización: 21
2.3 HTML5 21
2.3.1 Semántica web y soporte nativo: 22
2.4 CSS3 22
2.4.1 Diseño adaptativo y maquetación moderna: 23
2.5 Tailwind CSS 23
2.5.1 Compilación Just-in-Time (JIT) y mantenibilidad: 24
2.6 JavaScript 24
2.6.1 Asincronía y manipulación del DOM: 25
2.7 Node.js 25
2.8 Bases de datos 26

2.9 PostgreSQL 29
2.9.1 Cumplimiento ACID y soporte multimodal: 29
2.10 SQLAlchemy (ORM) 30
2.10.1 Mapeo Objeto Relacional y capa de abstracción: 30
2.11 Validación de datos 31
2.11.1 Niveles de validación 31
2.12 Inyección SQL 32
2.12.1 Mecanismos de defensa en profundidad: 32
2.13 GitHub 32
2.13.1 Control de versiones colaborativo y CI/CD: 33
2.14 Render 33
2.14.1 Integración continua y servicios gestionados: 34
CAPÍTULO III. DISEÑO Y MODELADO 35
3.1 Arquitectura del proyecto 36
3.2 Diagrama entidad relación 37
3.3 Diccionario de datos 38
3.4 Diagrama de casos de uso UML 41
3.5 Diagrama de flujo de proceso, ingreso a la plataforma 43
3.5.2 Administración del inventario 45
3.6 Modelado 46
3.6.1 Maquetado del primer módulo, difusión del hospital. 46
CAPÍTULO IV. DESARROLLO 51
4.1. Instalación de software. 52
4.1.1 Node.js (LTS): 52
4.2 PostgreSQL: 52
4.2.1 Programa pgAdmin4: 53
4.3. Instalación de Visual Studio Code 53
4.4. Estructura del proyecto. 54
4.5. Inicialización del entorno Node.js y dependencias. 54
4.5.1. Creación del archivo de configuración del proyecto: 54
4.5.2 Instalación de dependencias de producción backend y servicios: 54

4.5.3. Instalación de dependencias de desarrollo (Tailwind CSS): 55
4.6. Configuración y Compilación de Tailwind CSS 55
4.6.1. Inicializar el archivo de configuración: 55
4.7. Creación y edición de tailwind.config.js: 55
4.8. Creación del archivo base input.css con las directivas: 55
4.9. Creación de la Base de Datos. 56
4.10. Configuración de variables de entorno (.env). 57
4.11. Conexión de la base de datos en Node.js. 57
4.11.1. Comando para la activación del servidor. 58
CONCLUSIONES LIX
REFERENCIAS LX
ANEXOS LXII
Cronograma de actividades LXII
Requerimientos LXIII

ÍNDICE DE FIGURAS

Figura 1 Inicios del Hospital Mazahua 14
Figura 2 Hospital Mazahua 15
Figura 3 Ubicación del Hospital Mazahua 16
Figura 4 Logotipo del Hospital Mazahua 17
Figura 5 Logo de Hospitales COFAS 17
Figura 6 Organigrama general 18
Figura 7 Logo de Figma 20
Figura 8 Logo de Visual Studio Code 21
Figura 9 Logo de GTML5 22
Figura 10 Logo de CSS3 23
Figura 11 Logo de Tailwind CSS 24
Figura 12 Logo de JavaScript 25
Figura 13 Logo de Node JS 26
Figura 14 Logo de PostgreSQL 30
Figura 15 Logo de SQLAlchemy 31
Figura 16 Imagen Ilustrativo de Inyección SQL 32
Figura 17 Logo de GitHub 33
Figura 18 Logo de Render 33
Figura 19 Arquitectura del proyecto 36
Figura 20 Diagrama entidad relación 37
Figura 21 Diagrama de caso de usa, primera vista 41
Figura 22 Diagrama de caso de uso, segunda vista 42
Figura 23 Diagrama de flujo, ingreso a la plataforma 43
Figura 24 Diagrama de flujo, agendar una cita 44
Figura 25 Diagrama de flujo, administración del inventario 45
Figura 26 Pantalla de inicio 46
Figura 27 Pantalla de la información general del Hospital Mazahua 47
Figura 28 Pantalla de las instalaciones del Hospital Mazahua 48
Figura 29 Pantalla donde se consulta la ubicación y el contacto del Hospital Mazahua 49
Figura 30 Instalador de Node.js 52
Figura 31 Página oficial de PostgreSQL 52
Figura 32 Programa pgAdmin4 53
Figura 33 Instalador de Visual Studio Code 53

Figura 34 Estructura del proyecto 54
Figura 35 Código de tailwind.config.js 55
Figura 36 Código de configuración de CSS 55
Figura 37 Creación de la base de datos 56
Figura 38 Código SQL para la creación de la tabla usuarios 56
Figura 39 Verificación de la creación de la BD 56
Figura 40 Credenciales del archivo .env 57
Figura 41 Código para la conexión hacia la BD 57

ÍNDICE DE TABLAS

Tabla 1 Tipos de bases de datos 27
Tabla 2 Tabla de usuario 38
Tabla 3 Tabla de paciente 38
Tabla 4 Tabla cita 39
Tabla 5 Tabla de medicamento 39
Tabla 6 Tabla lote de medicamento 40
Tabla 7 Tabla movimiento del inventario 40

INTRODUCCIÓN

Una plataforma integral, se define como un paquete integrado de servicios y herramientas de software que proporciona una base completa para el ciclo de vida de una aplicación o sistema, diseñado para eliminar la complejidad de la infraestructura subyacente y permitir que los equipos se centren en la lógica y estrategia del proyecto en lugar de en la gestión individual de servidores, bases de datos y redes. (Google Cloud, s.f.).

En la actualidad, las tecnologías de la información resultan indispensables para modernizar la gestión en los servicios de salud. La adopción de sistema integral centraliza la información, reduce errores y optimiza el uso de recursos, facilitando procesos clave como el control de inventarios farmacéuticos y la programación de consultas.

En el Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua), las tareas de registro de medicamentos y asignación de citas médicas continúan realizándose de manera manual o mediante hojas de cálculo aisladas. Esta dinámica provoca desajustes en las existencias de fármacos, tiempos de espera prolongados y dificultades para obtener información administrativa en tiempo real. Para resolver esta problemática, el presente proyecto propone el desarrollo de una plataforma integral que digitalice el inventario, organice la disponibilidad de médicos y citas, y genere reportes útiles para la toma de decisiones.

El documento se encuentra organizado en las siguientes secciones: Planteamiento del Problema, Justificación y Objetivos: expone las deficiencias detectadas, la viabilidad de la propuesta y las metas técnicas del sistema.

Capítulo I. Generalidades de la Institución: describe los antecedentes, ubicación, filosofía institucional (misión, visión, valores) y estructura organizacional del Hospital Mazahua.

Capítulo II. Marco Teórico: reúne las bases conceptuales y herramientas tecnológicas aplicadas en el desarrollo de la solución.

Capítulo III. Desarrollo: se detalla el diseño, la arquitectura del software, la base de datos y la codificación de los módulos implementados.

Capítulo IV. Pruebas y Resultados: muestra las pruebas de funcionamiento de la plataforma y la validación de sus características operativas. Conclusiones y Referencias: resume los alcances obtenidos frente a los objetivos iniciales y lista las fuentes consultadas durante el proyecto.

PLANTEAMIENTO DEL PROBLEMA

El Centro de Orientación Familiar y Salud, I.A.P. (Hospital Mazahua) desempeña un papel fundamental en la atención médica de su comunidad, brindando servicios esenciales a una población que requiere atención constante y oportuna. Sin embargo, en el ámbito operativo interno, diversas tareas clave, el control del inventario de medicamentos y la programación de citas médicas continúan realizándose de manera manual y descentralizada, apoyándose en registros físicos en papel o herramientas de ofimática.

Esta dependencia de métodos tradicionales incrementa el riesgo de errores en la captura de datos, propicia la duplicidad de información y dificulta conocer en tiempo real las existencias y fechas de caducidad de los fármacos. Asimismo, la falta de un sistema automatizado para coordinar los horarios y la disponibilidad del personal médico genera retrasos en la asignación de turnos, tiempos de espera prolongados para los pacientes y una sobrecarga administrativa que limita la toma de decisiones ágiles y fundamentadas en la institución. A partir de este problema que esta presente en el Hospital Mazahua, surge la siguiente pregunta:

¿La implementación de una plataforma integral para la gestión de medicamentos y la programación de citas optimizará los procesos administrativos y la calidad de la atención al paciente en el Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua)?

JUSTIFICACIÓN

En las instituciones dedicadas a la salud, contar con procesos ágiles y ordenados resulta indispensable para ofrecer un servicio digno y continuo. Actualmente, en el Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua), actividades esenciales como el control del inventario en farmacia y la programación de citas médicas se realizan de forma manual o mediante hojas de cálculo. Esta dependencia de métodos tradicionales ocasiona lentitud en el registro de datos, desconocimiento en tiempo real sobre las existencias de insumos, duplicidad de información y demoras en la atención directa a los usuarios, elevando la posibilidad de errores administrativos que afectan la operación hospitalaria.

El desarrollo de un sistema integral modernizará y centralizará estos procedimientos operativos. Con la implementación del sistema se logrará automatizar el registro de entradas, salidas, existencias y fechas de caducidad de los medicamentos, lo que evitará pérdidas económicas por vencimiento de productos y prevendrá desabastos imprevistos. En el área de consultas, la plataforma permitirá organizar de manera estructurada los horarios, consultorios y disponibilidad de los especialistas, eliminando el empalme de citas, disminuyendo los tiempos de espera y garantizando una atención más puntual y ordenada para los pacientes de la comunidad.

Asimismo, otro beneficio sustancial será la generación automática de reportes estadísticos e indicadores administrativos. Al concentrar los movimientos en un solo lugar, el personal directivo dispondrá de información confiable sobre la demanda real de consultas por especialidad y el consumo histórico de fármacos. Esta herramienta facilitará la toma de decisiones estratégicas, mejorará la planeación de compras y permitirá supervisar el rendimiento institucional sin recurrir a revisiones manuales complejas.

La viabilidad del proyecto está plenamente respaldada en los aspectos técnico, operativo y económico. En lo técnico, se emplearán tecnologías web estándar, seguras y de código abierto que no requieren licencias costosas ni servidores complejos, funcionando directamente desde los navegadores de los equipos ya existentes en la unidad médica. En el ámbito operativo, el diseño intuitivo y accesible de las pantallas facilitará una rápida adaptación por parte del personal médico y administrativo, sin interrumpir las labores diarias, su viabilidad económica es clara, ya que aprovecha la infraestructura tecnológica disponible en la institución para resolver una necesidad prioritaria con una inversión sumamente baja y un alto beneficio comunitario.

OBJETIVOS

General:

Desarrollar una plataforma integral para la gestión de medicamentos y generación de citas del Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua), mediante la digitalización y centralización de la información, con el fin de optimizar los procesos administrativos y mejorar la atención al paciente.

Específicos:

- Controlar las entradas, salidas, existencias y caducidades de medicamentos.
- Administrar la disponibilidad de médicos, horarios y citas médicas.
- Generar reportes que faciliten la toma de decisiones administrativas.
- Validar el funcionamiento de la plataforma mediante pruebas de los módulos implementados.

CAPÍTULO I. GENERALIDADES DE LA INSTITUCIÓN

Describe el contexto del Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua), abarcando sus antecedentes históricos, ubicación geográfica, filosofía organizacional (misión, visión y valores), logotipos de la institución y organigrama general.

1.1 Antecedentes

El Hospital Mazahua es fundado en el año 1973, cuando la Asociación Mexicana Albert Schweitzer inició un proyecto en el municipio de San Felipe del Progreso para impulsar el desarrollo integral de la región Mazahua. Como parte de este proyecto se construyó el Dispensario Mazahua, el cual fue inaugurado en enero de 1975.

Posteriormente, en 1994, la Asociación Programa Compartamos recibió en donación el proyecto que actualmente corresponde al Hospital Mazahua. A partir de entonces se inició su transformación en un hospital especializado en la atención materno-infantil, incorporando nuevos servicios y aumentando el personal para atender las necesidades de la población. (García, 2021, p. 45)



Figura 1 Inicios del Hospital Mazahua

Recuperado de: https://share.google/G2dOF10rvxwauJxlu

El Hospital Mazahua es una unidad médica de segundo nivel perteneciente a los Centros de Orientación Familiar y Salud, I. A. P. (COFAS), cuya misión primordial radica en proporcionar servicios de salud accesibles, oportunos y de alta calidad técnica y humana para fortalecer el bienestar integral de la población en su zona de influencia. Como eje asistencial de referencia en la región, la institución responde a una demanda continua mediante una oferta multidisciplinaria estructurada para atender tanto episodios críticos como padecimientos crónicos, contando para ello con servicios esenciales ininterrumpidos como atención de urgencias y farmacia comunitaria las 24 horas del día. (García, 2021, p. 45)

Su cartera asistencial integra unidades diagnósticas de laboratorio de análisis clínicos, gabinete de imagenología con rayos X y ultrasonido, así como áreas clínicas de hospitalización continua para pacientes pediátricos y adultos, procedimientos quirúrgicos programados y de urgencia, atención obstétrica integral y partos. Asimismo, el área de consulta externa dispone de una sólida gama de especialidades que abarca medicina general, ginecología y obstetricia, cirugía general, pediatría, urología, medicina interna, cardiología, angiología, traumatología y ortopedia, odontología, psicología, nutrición, optometría y un enfoque distintivo de salud intercultural adaptado al entorno social comunitario. (CMH, s. f., sección de Especialidades)



Figura 2 Hospital Mazahua

Recuperado de: https://share.google/9Lvx8c9Zn2JQaOUpC

El Hospital Mazahua opera con una infraestructura física e instalada diseñada para satisfacer la demanda de atención médica primaria y especializada. La capacidad censable y de estancia incluye 17 camas de hospitalización general, 2 cuneros para la atención y monitorización neonatal, 1 quirófano completamente equipado para intervenciones quirúrgicas de diversas complejidades y 1 sala de expulsión exclusiva para la atención segura del trabajo de parto. A este esquema resolutivo se suman el módulo permanente de urgencias médicas, la farmacia de servicio continuo, las áreas analíticas y de imagen diagnóstica, junto con 10 consultorios médicos habilitados para el desahogo fluido de la consulta externa. (CMH, s. f., sección de Especialidades)

1.2 Ubicación.

El Hospital Mazahua se encuentra ubicado en Santa Ana Nichi, municipio de San Felipe del Progreso, Estado de México.

Domicilio: Carretera a San Pedro el Alto, km 10, Santa Ana Nichi, C.P. 50686, San Felipe del Progreso, México.

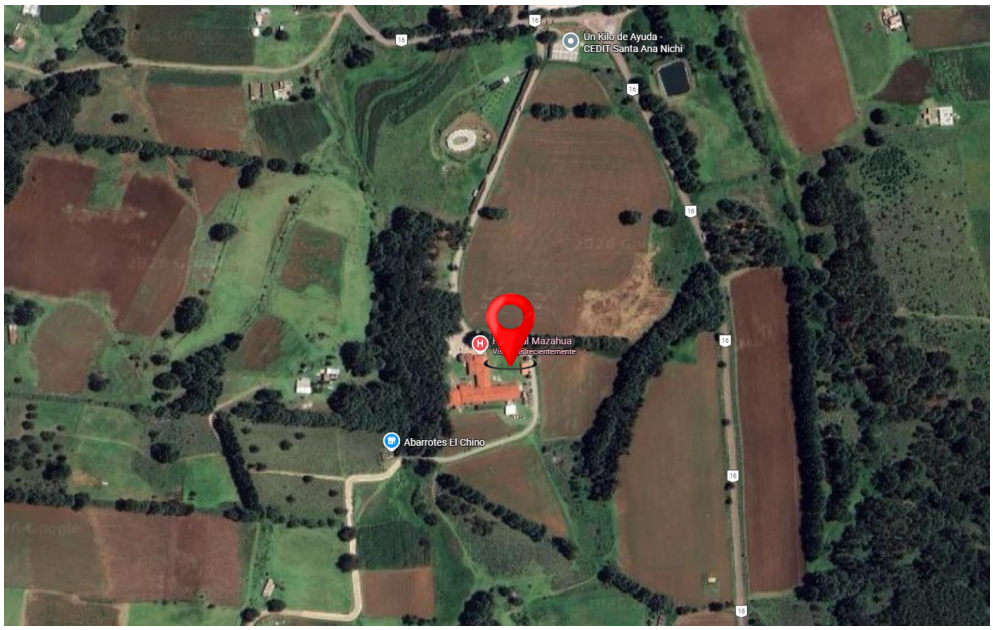


Figura 3 Ubicación del Hospital Mazahua

Recuperado de: Google Maps

1.3 Misión

Proporcionar servicios de salud integrales y especializados, a través de un sistema de clínicas y hospitales autosuficientes en operación, y que nuestros servicios se brinden con calidad, trato digno y valor agregado a la población que carece de ellos.

1. 4 Misión:

Generar bienestar en las comunidades, a través de un sistema innovador de servicios integrales de salud.

Ser el sistema modelo a nivel nacional en prestación de servicios de salud para millones de personas que viven en condiciones de vulnerabilidad.

1.5 Valores:

- **Persona:** Toda persona es única e irrepetible, reconocemos su dignidad sin distinción alguna.

- **Servicio:** Somos sensibles a las necesidades de los demás; encontramos en el servicio nuestra razón de ser.
- **Compasión:** Basados en el amor y la empatía vivimos un profundo deseo de bienestar a los demás actuando sin juicio ni crítica.
- **Colaboración:** Nuestras acciones van encaminadas a la cooperación, trabajando en orden y con comunicación.
- **Lealtad:** Construimos relaciones interpersonales sólidas, basadas en el respeto, el honor y la gratitud.
- **Responsabilidad:** Cumplimos nuestros deberes con excelencia y asumimos las consecuencias de nuestras acciones.
- **Conocimiento:** Vivimos este valor, privilegiando el análisis y la reflexión buscando en cada acción alcanzando la excelencia.

1.6 Logotipos de la institución.

Las siguientes imágenes son las ilustraciones oficiales del hospital que se utilizan para su difusión, la figura 4 es del hospital donde se realiza la residencia y la figura 5 es parte de la organización que dirige a los diferentes centros de salud de la zona.



Figura 4 Logotipo del Hospital Mazahua

Recuperado de: https://share.google/6u6yL1Xy0szsl7rG1



*F*igura 5 Logo de Hospitales COFAS

Recuperado de: https://share.google/WRnDxWMDPBguWDF5l

1.7 Organigrama general.

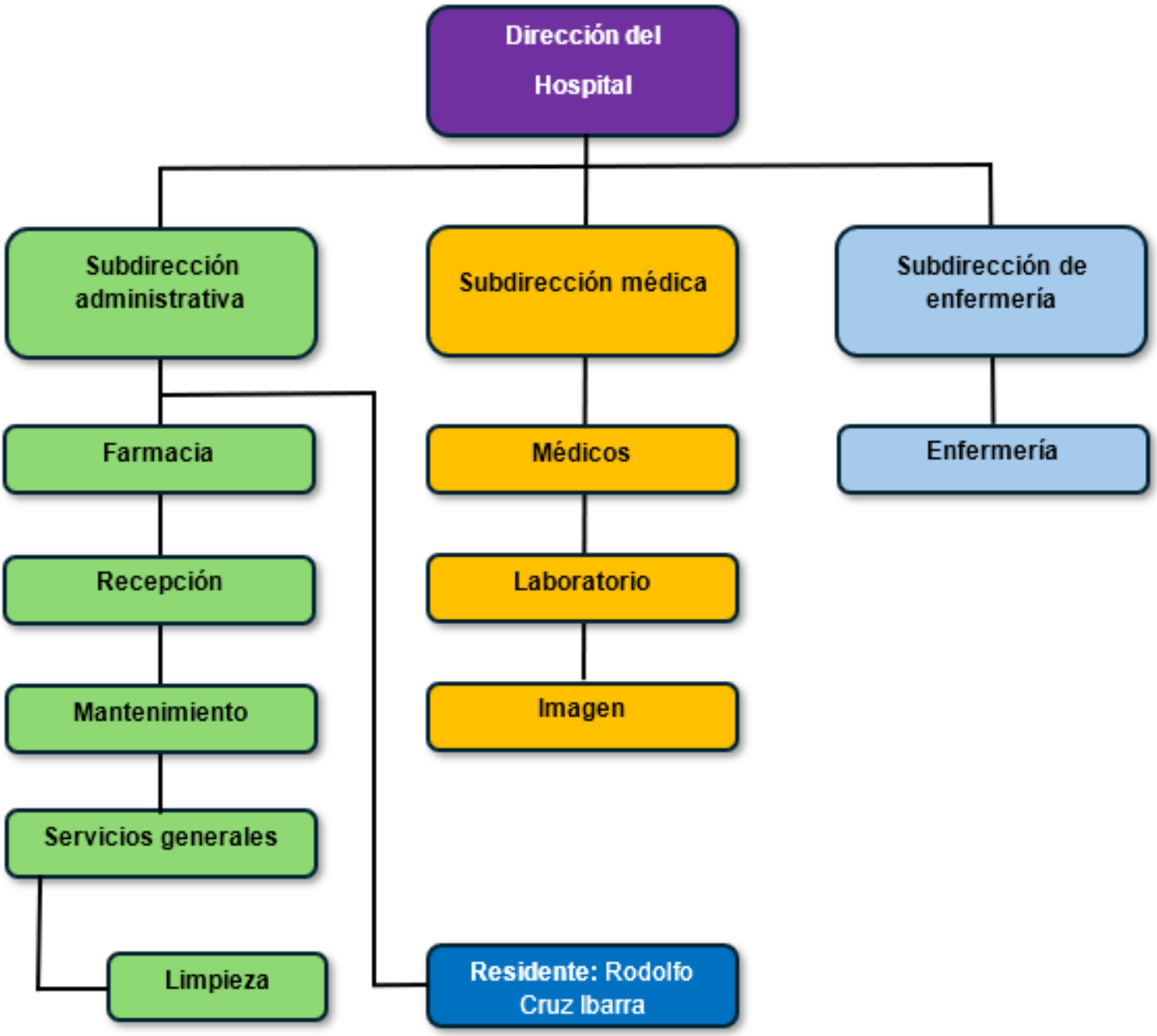


Figura 6 Organigrama general

Elaboración propia

CAPÍTULO II. MARCO TEÓRICO

Se expone los fundamentos conceptuales, normativos y tecnológicos que sustentan el proyecto, analizando los estándares de calidad en salud, las metodologías de desarrollo de software y las herramientas web empleadas.

2.1 Figma

Figma es una herramienta de diseño de interfaces y prototipado, basada en la nube, que permite crear diseños de sitios web y aplicaciones, así como prototipos interactivos navegables, sin necesidad de instalar software adicional. Una de sus características distintivas es la colaboración en tiempo real, que permite que varios usuarios trabajen simultáneamente sobre un mismo archivo, además de ofrecer componentes y estilos reutilizables que facilitan mantener la coherencia visual a lo largo de un proyecto. (Lupton, 2020).



Figura 7 Logo de Figma

Recuperado de: https://share.google/Pex9Z83R8BDiUZJko

2.1.1 Sistemas de diseño y traspaso a desarrollo:

La plataforma implementa propiedades visuales análogas a CSS como *Auto Layout*, lo que permite diseñar componentes elásticos que simulan con precisión el comportamiento de interfaces web adaptables (Figma, 2023). Incluye paneles de inspección que exponen medidas, fuentes y propiedades estilísticas a los programadores, reduciendo la brecha técnica en la transición del diseño conceptual a la implementación de código (Lupton, 2020). Figma es un entorno colaborativo en la nube donde se define la arquitectura visual, la usabilidad y la experiencia de usuario antes de iniciar la programación. Los sistemas de diseño basados en componentes permiten concebir interfaces dinámicas que replican con exactitud el comportamiento que tendrá el código final ante distintos tamaños de pantalla. Esto agiliza la toma de decisiones, previene retrabajos durante la codificación y facilita la consulta de medidas, márgenes y paletas de colores exactas al momento de maquetar.

2.2 Visual Studio Code

Visual Studio Code es un editor de código fuente ligero pero potente, desarrollado por Microsoft, disponible para Windows, macOS y Linux. Ofrece soporte nativo para JavaScript, TypeScript y Node.js y a través de un amplio ecosistema de extensiones puede ampliarse para trabajar con prácticamente cualquier lenguaje de programación o entorno de

ejecución. Entre sus características principales se encuentran el resaltado de sintaxis, el autocompletado inteligente de código, la depuración integrada y el soporte para control de versiones mediante Git. (Sommerville, 2016).

2.2.1 Protocolo de lenguaje (LSP) y personalización:

El núcleo del editor se apoya en el *Language Server Protocol* (LSP), permitiendo integrar análisis estático, detección de sintaxis errónea y refactorización inteligente independiente del lenguaje utilizado (Microsoft, 2023). Su consola de depuración y terminal integrada posibilitan interactuar con entornos virtuales y ramas de control de versiones dentro de la misma ventana de trabajo (Sommerville, 2016).

Visual Studio Code es el entorno integrado de desarrollo sobre el cual se centraliza la construcción del software. Su ligereza combinada con su arquitectura basada en extensiones permite adaptarlo a los lenguajes del proyecto (HTML, JavaScript, Python y CSS). Además, al integrar la consola de comandos, el depurador de código y el control de ramas de Git en una sola ventana, optimiza el flujo de trabajo diario del desarrollador.



Figura 8 Logo de Visual Studio Code

Recuperado de: https://share.google/41Cnf5vBkyjx9Hxg4

2.3 HTML5

HTML5 es la última versión de HyperText Markup Language (Lenguaje de Marcado de Hipertexto). Combina tres tecnologías web: HTML, para construir la estructura de la página; CSS, para mejorar la presentación; y JavaScript, para agregar funcionalidad a los elementos HTML. Sus etiquetas o marcas hacen que textos, imágenes y elementos multimedia se visualicen correctamente en la pantalla, y que las páginas web tengan un formato unificado, estandarizado y sean compatibles con distintos navegadores, como Chrome, Edge, Firefox y Safari. (W3C, 2021).

2.3.1 Semántica web y soporte nativo:

La evolución hacia HTML5 introdujo etiquetas semánticas (<header>, <nav>, <main>, <article>, <section>, <footer>) que optimizan la indexación en motores de búsqueda (SEO) y facilitan la lectura de interfaces mediante tecnologías de asistencia (MDN Web Docs, 2023). Asimismo, prescinde de complementos externos de terceros al integrar soporte nativo para audio y video mediante los elementos <audio> y <video>, garantizando una arquitectura web abierta y estandarizada (W3C, 2021).

HTML5 constituye el esqueleto estructural y funcional indispensable de toda la aplicación web. No se limita únicamente a desplegar textos, botones o imágenes, sino que gracias a la incorporación de etiquetas semánticas como <header>, <nav>, <main> y <footer>, otorga un significado jerárquico y contextual claro a cada bloque de la interfaz. Esto resulta fundamental para optimizar el posicionamiento en motores de búsqueda y garantizar la accesibilidad digital para lectores de pantalla. Asimismo, al contar con soporte nativo para elementos multimedia mediante las etiquetas <audio> y <video>, elimina la necesidad de recurrir a complementos externos obsoletos, garantizando una interpretación limpia, rápida y estandarizada en cualquier navegador web moderno.



Figura 9 Logo de GTML5

Recuperado de: https://share.google/l8Txh8p39vZ0Zp8G2

2.4 CSS3

CSS3 es un lenguaje de hojas de estilo modularizado que amplía las capacidades de CSS2. Su propósito principal sigue siendo la separación del contenido del documento (estructura) de su presentación (estilo). Esta separación facilita el mantenimiento, mejora la accesibilidad y proporciona mayor flexibilidad en el diseño web.

La evolución de CSS a CSS3 implicó dividir la especificación en varios módulos independientes, cada uno de los cuales aborda áreas específicas de estilo, como selectores, colores, fondos, bordes, efectos de texto, animaciones y diseño. (W3C, 2023).

2.4.1 Diseño adaptativo y maquetación moderna:

Entre las incorporaciones más críticas de CSS3 destaca el uso de *Media Queries*, mecanismo que permite condicionar la visualización del contenido según la resolución y orientación de la pantalla, principio base del *Responsive Web Design* (Marcotte, 2011). Igualmente, resolvió los desafíos de alineación y distribución espacial mediante modelos de maquetación avanzados como Flexbox y CSS Grid, reduciendo la dependencia de reglas flotantes complejas (W3C, 2023).

CSS3 es la capa responsable de dotar de identidad visual, coherencia y dinamismo estético al proyecto, manteniendo una separación estricta respecto al contenido HTML. Esta división es clave en el desarrollo profesional, ya que permite ajustar colores, tipografías y márgenes sin afectar la lógica estructural de los archivos. Su aporte más significativo es la implementación de las *Media Queries*, Flexbox y CSS; estas herramientas permiten construir un diseño responsivo real, logrando que la interfaz se reestructure fluidamente según la resolución y orientación de monitores de escritorio, tabletas o dispositivos móviles sin depender de arreglos rígidos o desbordamientos visuales.



Figura 10 Logo de CSS3

Recuperado de: https://share.google/GePNaxYxkoxddQl5T

2.5 Tailwind CSS

Es un framework CSS que da prioridad a la utilidad sobre el propio estilo, pero además a diferencia de otros frameworks CSS como Bootstrap o Bulma, Tailwind no provee una serie de componentes predefinidos. En su lugar, este framework opera en un nivel inferior y proporciona un conjunto de clases de ayuda para estructura y estilado, de forma que, usando dichas clases, puedas crear rápidamente diseños personalizados con facilidad. Además, gracias a su flexibilidad permite crear un diseño realmente único.

2.5.1 Compilación Just-in-Time (JIT) y mantenibilidad:

Tailwind utiliza un motor de compilación Just-in-Time (JIT) que escanea los archivos del proyecto (HTML, JS o plantillas del servidor) y genera exclusivamente las reglas CSS que se están utilizando, eliminando hojas de estilo sobredimensionadas en entornos de producción (Wathan et al., 2023). Su enfoque basado en clases utilitarias agiliza el mantenimiento del código, impidiendo efectos secundarios en cascada no deseados al modificar vistas específicas.

Tailwind CSS transforma el enfoque clásico de maquetación al apoyarse en una metodología de clases de utilidad, en lugar de ofrecer componentes rígidos preensamblados como otros frameworks. Esto brinda total libertad para crear interfaces personalizadas y consistentes directamente desde el marcado HTML, sin saltar constantemente a hojas de estilo externas. Su gran ventaja técnica reside en el motor de compilación, el cual analiza en tiempo real el código fuente y genera exclusivamente el código CSS que realmente se utiliza, descartando estilos innecesarios y garantizando hojas de estilo finales livianas, optimizadas y con cargas de página casi instantáneas.



Figura 11 Logo de Tailwind CSS

Recuperado de: https://share.google/V0cGazyZ2VKXy2urN

2.6 JavaScript

JavaScript o JS es un lenguaje de programación diseñado para escribir las partes Front End (de la parte del cliente) y Back End (de la parte del servidor) de sitios web y aplicaciones móviles. Es un lenguaje de alto nivel, legible para los humanos, pero no requiere un compilador para traducirlo al lenguaje de máquina. El código se interpreta y se ejecuta directamente en un navegador web, lo que facilita y acelera la interacción con el usuario.

JavaScript es un lenguaje de scripts de secuencias de comandos. Permite agregar muchas funcionalidades a las páginas web haciéndolas más interactivas, útiles y atractivas para el usuario. Algunos ejemplos de contenido dinámico son menús desplegables, formularios, galerías de fotos, gráficos animados y más. (MDN Web Docs, 2024).



Figura 12 Logo de JavaScript

Recuperado de: https://share.google/T1A3c8GnYbbCHP6fS

2.6.1 Asincronía y manipulación del DOM:

JavaScript interactúa con el Document Object Model (DOM), permitiendo modificar la estructura, el estilo y el contenido de un documento de forma reactiva en tiempo real (Flanagan, 2020). Con la integración de la API Fetch y el paradigma asíncrono basado en promesas y sintaxis async/await, el lenguaje posibilita el envío y recepción de paquetes de datos en segundo plano sin forzar la recarga total de la página web (MDN Web Docs, 2024).

JavaScript es el lenguaje que otorga vida, reactividad e interactividad al sistema directamente en el navegador del cliente. Su capacidad para manipular en tiempo real el Modelo de Objetos del Documento (DOM) permite modificar dinámicamente elementos de la pantalla, mostrar notificaciones, validar formularios y responder de inmediato a los clics o movimientos del usuario. Además, con la integración de la API Fetch y el manejo de flujos asíncronos mediante async/await, el sistema puede comunicarse con el servidor e intercambiar datos en formato JSON en segundo plano, evitando la necesidad de recargar la página completa y brindando una experiencia de uso sumamente fluida.

2.7 Node.js

Node.js es un entorno de ejecución (*runtime environment*) para JavaScript multiplataforma y de código abierto, diseñado para ejecutar código fuera de un navegador web, principalmente en la capa del servidor (*Back-End*). Construido sobre el motor V8 de Google Chrome, emplea un modelo de entrada/salida (I/O) sin bloqueo y dirigido por eventos (*event-driven*), lo que le confiere una alta eficiencia y ligereza para gestionar múltiples conexiones de forma concurrente con un bajo consumo de recursos (Casciaro & Mammino, 2020). Asimismo, cuenta con el gestor de paquetes npm (*Node Package Manager*), el cual

provee acceso a un extenso ecosistema de módulos y dependencias de código abierto reutilizables (OpenJS Foundation, 2023).

2.7.1 Bucle de eventos (*Event Loop*) y arquitectura asíncrona:

La característica arquitectónica central de Node.js radica en su bucle de eventos de un solo hilo (*single-threaded event loop*), el cual delega las operaciones costosas del sistema operativo (como consultas a disco, conexiones de red o acceso a bases de datos) a subprocesos en segundo plano a través de la biblioteca libuv (Tilkov & Vinoski, 2010). Este mecanismo permite procesar miles de solicitudes de red simultáneas sin incurrir en la sobrecarga de memoria asociada a la creación de múltiples hilos tradicionales, posicionándolo como una solución idónea para servicios web orientados a eventos, APIs REST y flujos de datos en tiempo real (Cantelon et al., 2017).

Node.js amplía las capacidades de JavaScript al trasladarlo del entorno restringido del navegador hacia el lado del servidor y la consola de desarrollo. Su principal fortaleza no reside en la velocidad de cálculo matemático puro, sino en su modelo de concurrencia basado en el *Event Loop*: al no bloquear la ejecución mientras espera respuestas de lectura de archivos o consultas externas, puede procesar miles de peticiones simultáneas utilizando un único hilo principal sin colapsar la memoria del servidor.



Figura 13 Logo de Node JS

Recuperado de: https://share.google/1Y8Ti3NVY9CCIYpyJ

2.8 Bases de datos

Una base de datos es una colección organizada, estructurada y coherente de datos interrelacionados, diseñada para facilitar su almacenamiento, consulta, actualización y administración eficiente mediante un Sistema de Gestión de Bases de Datos (DBMS, por sus siglas en inglés) (Elmasri & Navathe, 2017). A diferencia de los esquemas convencionales de almacenamiento en archivos planos, una base de datos garantiza la independencia entre los datos lógicos y los programas que los consumen, reduce la

redundancia al mínimo, aplica mecanismos de seguridad por roles y preserva la integridad y consistencia de la información ante fallos del sistema o accesos concurrentes (Silberschatz et al., 2020).

Las bases de datos el pilar organizativo y conceptual que sostiene la persistencia de toda aplicación informática. A diferencia del almacenamiento en archivos independientes o planos, una base de datos centraliza la información de forma estructurada e interrelacionada bajo la supervisión de un software. Esto garantiza la independencia entre los programas y los datos físicos, erradica la duplicidad innecesaria, facilita consultas analíticas eficientes y asegura que la información permanezca intacta, consistente y disponible para los usuarios autorizados.

Tabla 1 Tipos de bases de datos

Tipo de base de datos	Descripción y arquitectura	Beneficios principales	Ejemplos representativos
Relacionales (RDBMS / SQL)	Organizan la información en tablas compuestas por filas y columnas vinculadas mediante llaves primarias y foráneas (Coronel & Morris, 2019).	• Integridad transaccional estricta mediante propiedades ACID. • Estandarización mediante lenguaje declarativo SQL. • Prevención de inconsistencias y duplicidad de registros (Silberschatz et al., 2020).	PostgreSQL, MySQL, Oracle Database, SQLite.
Documentales (NoSQL)	Almacenan registros semiestructurados como documentos (JSON, BSON o XML), donde cada registro puede contener pares clave-	• Esquema dinámico sin rigidez estructural. • Alta velocidad para lectura, escritura de objetos complejos.	MongoDB, CouchDB.

	valor anidados y listas (Sadashiv & Kumar, 2011).	• Facilidad de mapeo directo con objetos de lenguajes modernos (Sadalage & Fowler, 2013).	
Clave-Valor (NoSQL)	Modelo minimalista donde cada ítem consta de una clave única vinculada a un valor binario o de texto (Sadalage & Fowler, 2013).	• Latencia extremadamente baja (acceso en memoria). • Escalabilidad horizontal casi lineal. • Ideal para gestión de sesiones, carritos de compra y caché rápida.	Redis, Amazon DynamoDB, Memcached.
Orientadas a Columnas / Familias de Columnas	Almacenan y gestionan los datos agrupados por columnas físicas contiguas en lugar de filas (Celesti et al., 2019).	• Compresión de datos muy eficiente al agrupar tipos similares. • Alto rendimiento en agregaciones y consultas analíticas sobre grandes volúmenes (*Big Data*).	Apache Cassandra, Apache HBase.
Grafos (NoSQL)	Estructuradas en nodos (entidades), aristas (relaciones dirigidas) y propiedades (atributos), priorizando las conexiones entre datos (Robinson et al., 2015).	• Consultas de redes complejas con rendimiento constante. • Excelente para análisis de fraude, redes sociales y motores de recomendación.	Neo4j, Amazon Neptune.

		• Eliminación de operaciones de unión (*JOIN*) costosas.	
Series de Tiempo (Time-Series)	Diseñadas para registrar y ordenar secuencialmente flujos continuos de datos indexados por marcas de tiempo (*timestamps*) (Derakhshan et al., 2019).	• Optimización de escritura y lectura temporal masiva. • Algoritmos integrados de retención, compresión y muestreo temporal. • Ideal para telemetría, métricas de servidores y sensores IoT.	InfluxDB, TimescaleDB, Prometheus.

2.9 PostgreSQL

es un sistema de gestión de bases de datos relacionales de código abierto que almacena, organiza y recupera datos estructurados. Esta base de datos relacional impone relaciones entre las tablas de datos, valida los datos a medida que ingresan a los sistemas y mantiene la precisión de los datos incluso cuando varios usuarios realizan cambios simultáneamente. Las organizaciones eligen PostgreSQL cuando necesitan una base de datos relacional confiable y que cumpla con los estándares, sin costos de licencia. (Riggs et al., 2018).

2.9.1 Cumplimiento ACID y soporte multimodal:

PostgreSQL destaca por su estricta adherencia a las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), garantizando la fidelidad e integridad transaccional frente a concurrencia elevada o fallos de energía (PostgreSQL Global Development Group, 2023). Asimismo, maneja tipos de datos semiestructurados como JSON y JSONB con soporte de indexación especializada, combinando la solidez relacional tradicional con la agilidad de los almacenes documentales (Riggs et al., 2018).

PostgreSQL es el motor de base de datos relacional encargado de la persistencia segura, ordenada y consistente de toda la información transaccional del sistema. Su cumplimiento riguroso de sus propiedades, asegura que cada operación se complete de forma confiable, evitando inconsistencias o duplicidad de registros frente a accesos masivos concurrentes o fallas de red. Su capacidad para combinar modelos relacionales estrictos mediante llaves

primarias y foráneas, junto con el almacenamiento y consulta de datos semiestructurados que ofrece robustez técnica sin costos de licenciamiento.



Figura 14 Logo de PostgreSQL

Recuperado de: https://share.google/kyGF4rEIadMJrEo93

2.10 SQLAlchemy (ORM)

SQLAlchemy es el conjunto de herramientas SQL de Python que permite a los desarrolladores acceder y gestionar bases de datos SQL con el lenguaje de dominio Python. Puedes escribir una consulta en forma de cadena o encadenar objetos Python para consultas similares. Trabajar con objetos proporciona flexibilidad a los desarrolladores y les permite crear aplicaciones de alto rendimiento basadas en SQL.

Permite a los usuarios conectar bases de datos utilizando lenguaje Python, ejecutar consultas SQL mediante programación basada en objetos y agilizar el flujo de trabajo. (Grinberg, 2018).

2.10.1 Mapeo Objeto Relacional y capa de abstracción:

El componente ORM (Object-Relational Mapping) de SQLAlchemy traduce tablas y relaciones relacionales a clases y atributos de Python mediante el patrón *Data Mapper*, desacoplando la estructura de las entidades de la persistencia de datos (Bayer, 2022). Esto permite cambiar de motor de base de datos relacional con mínimas modificaciones en el código y automatiza la generación segura de esquemas y transacciones (Grinberg, 2018).

SQLAlchemy actúa como un puente de abstracción fundamental entre el lenguaje de programación Python y el motor relacional de la base de datos. Traduce las tablas y registros en clases y objetos de Python, permitiéndome interactuar con los datos a través de métodos de programación orientada a objetos en lugar de redactar sentencias SQL puras en cadenas de texto. Esto no solo desacopla la aplicación del motor de base de datos específico (lo que facilitaría una migración técnica en el futuro), sino que previene fallos humanos y parametriza de forma automática las operaciones de persistencia.



Figura 15 Logo de SQLAlchemy

Recuperado de: https://share.google/LyrBr7ODoGMbfmaro

2.11 Validación de datos

La validación de datos es el proceso crítico dentro de la ingeniería de software y la gestión de datos encargado de verificar que los datos ingresados por un usuario, una interfaz de programación (API) o un servicio externo cumplan con reglas predefinidas de exactitud, formato, completitud y consistencia antes de ser procesados o almacenados de manera permanente en el sistema (Pressman & Maxim, 2020). Su objetivo fundamental es asegurar la integridad del dominio de datos, evitar la corrupción del estado de la aplicación y prevenir fallos lógicos derivados de información mal formada.

2.11.1 Niveles de validación

La validación eficaz se estructura bajo un enfoque de defensa en profundidad distribuido en dos capas principales (Sommerville, 2016):

Validación en el cliente (Front-End): Se implementa en la interfaz de usuario mediante atributos nativos de HTML5 (como required, pattern, type="email") y lógica reactiva en JavaScript. Su propósito es brindar retroalimentación inmediata al usuario sobre campos obligatorios o formatos erróneos, reduciendo peticiones innecesarias al servidor y mejorando la experiencia de usuario (MDN Web Docs, 2023).

Validación en el servidor (Back-End): Constituye el filtro de seguridad obligatorio de la arquitectura. Dado que las restricciones del navegador pueden ser eludidas o manipuladas fácilmente mediante herramientas de red o clientes HTTP automatizados, el servidor debe evaluar nuevamente los tipos de datos, longitudes, rangos y reglas de negocio críticas antes de interactuar con la base de datos o ejecutar lógica de persistencia (OWASP, 2023).

La validación de datos es la técnica obligatoria para garantizar la integridad, consistencia y fiabilidad de toda la información que procesa el sistema. Debe concebirse en dos niveles complementarios: por un lado, la validación en el cliente, que utiliza atributos de HTML5 y JavaScript para guiar al usuario de forma amigable y prevenir envíos con campos vacíos o formatos equivocados; por otro lado, la validación en el servidor, que actúa como la barrera infranqueable de seguridad donde se confirman longitudes, rangos y reglas de negocio, protegiendo al sistema contra peticiones manipuladas o maliciosas.

2.12 Inyección SQL

La inyección SQL es descrita como una vulnerabilidad de seguridad que ocurre cuando una aplicación construye consultas SQL de forma dinámica a partir de datos proporcionados por el usuario sin validarlos adecuadamente, lo que permite a un atacante insertar código malicioso capaz de leer, modificar o eliminar información de la base de datos sin autorización. La técnica de prevención más recomendada es el uso de consultas preparadas o parametrizadas, las cuales separan la estructura de la consulta SQL de los datos ingresados por el usuario, de modo que estos últimos nunca se interpretan como código ejecutable. (OWASP, 2023).

2.12.1 Mecanismos de defensa en profundidad:

El estándar de la Open Web Application Security Project (OWASP, 2021) sitúa las fallas de inyección entre los riesgos críticos de seguridad de software web. Además del uso mandatorio de parámetros vinculados (*bound parameters*) y ORMs, se complementa con el principio de mínimo privilegio en los roles de conexión de base de datos y la validación/desinfección rigurosa de entradas en el servidor (OWASP, 2023).

La inyección SQL es una de las vulnerabilidades más peligrosas en la seguridad de aplicaciones web, la cual ocurre cuando datos maliciosos no saneados introducidos por un usuario se incorporan dinámicamente dentro de la consulta SQL ejecutada por el motor. Esto puede conceder al atacante la capacidad de sustraer, alterar o destruir por completo información confidencial.



Figura 16 Imagen Ilustrativo de Inyección SQL

Recuperado de: https://share.google/mqQTvKT4uXKaoxCPJ

2.13 GitHub

GitHub es una herramienta esencial para los ingenieros de software que ayuda a mejorar el flujo de trabajo y promueve la colaboración. 94 millones de desarrolladores, 4 millones de organizaciones y 330 millones de repositorios forman parte de la plataforma, y la comunidad de GitHub sigue creciendo. Esta popularidad se debe a la gran cantidad de

opciones disponibles para controlar y gestionar proyectos de código en cualquier lenguaje de programación o sistema operativo. (GitHub, 2023).

2.13.1 Control de versiones colaborativo y CI/CD:

Al apoyarse en la arquitectura distribuida de Git, GitHub permite la ramificación (*branching*) y combinación (*merging*) concurrente de código sin conflictos directos, estructurando revisiones entre pares a través de *Pull Requests* (Chacon & Straub, 2014). Su módulo integrado de GitHub Actions facilita automatizar pruebas unitarias, compilaciones e implementaciones en ambientes de prueba o producción (GitHub, 2023).

GitHub es la plataforma central que estructura el trabajo colaborativo, la trazabilidad y la preservación del código fuente apoyándose en la tecnología de Git. Permite registrar de forma exacta cada modificación del proyecto, crear ramas independientes para implementar nuevas funcionalidades sin comprometer la estabilidad del sistema principal, y consolidar cambios de forma controlada mediante *Pull Requests*.



Figura 17 Logo de GitHub

Recuperado de: https://share.google/Nq6qGnGz4BskjXVHK

2.14 Render

Render es definido como una plataforma en la nube de tipo PaaS (Platform as a Service), que permite alojar y desplegar aplicaciones web, APIs, servicios y sitios estáticos, automatizando gran parte de las tareas de configuración de infraestructura. Entre sus funcionalidades se encuentran el despliegue automatizado a partir de repositorios de control de versiones, la emisión de certificados de seguridad, y el soporte para lenguajes como Python. (Render, 2023).



Figura 18 Logo de Render

Recuperado de: https://share.google/ApjRScL1pz3AqZZet

2.14.1 Integración continua y servicios gestionados:

La plataforma implementa flujos de estrategia continúa activando compilaciones automáticas cada vez que se detectan cambios en ramas designadas del repositorio Git (Render, 2023). Integra además servicios de bases de datos gestionadas, variables de entorno aisladas y aprovisionamiento automático de certificados SSL/TLS mediante Let's Encrypt sin requerir configuraciones de red complejas (Render, 2023).

Render es la plataforma en la nube que agiliza la transición de la aplicación desde un entorno de desarrollo local hacia producción en internet. Automatiza la administración de infraestructura compleja: se vincula con el repositorio de control de versiones y activa despliegues continuos cada vez que se detectan cambios en el código. Además, simplifica enormemente la gestión operativa al aprovisionar automáticamente certificados de seguridad para una navegación segura, gestiona variables de entorno de forma confidencial y mantener activos los servicios web y la base de datos de manera coordinada.

CAPÍTULO III. DISEÑO Y MODELADO

Se presenta el diseño de la interfaz de usuario, el modelado de la base de datos, diagramas entidad relación, diccionario de datos, la representación visual de los procesos institucionales mediante diagramas de casos de uso y flujos del sistema.

3.1 Arquitectura del proyecto

Este diagrama representa la estructura técnica multinivel y el flujo de comunicación de la plataforma, modelando cómo la interfaz de usuario en HTML5, estilizada de forma responsiva con Tailwind CSS y dinamizada con JavaScript para alertas y animaciones, interactúa mediante peticiones web seguras con el servidor en la nube Render; este último ejecuta el entorno Node.js bajo JavaScript para procesar la lógica de negocio de citas e , comunicándose finalmente con la base de datos relacional PostgreSQL para asegurar la persistencia y consistencia de la información hospitalaria.

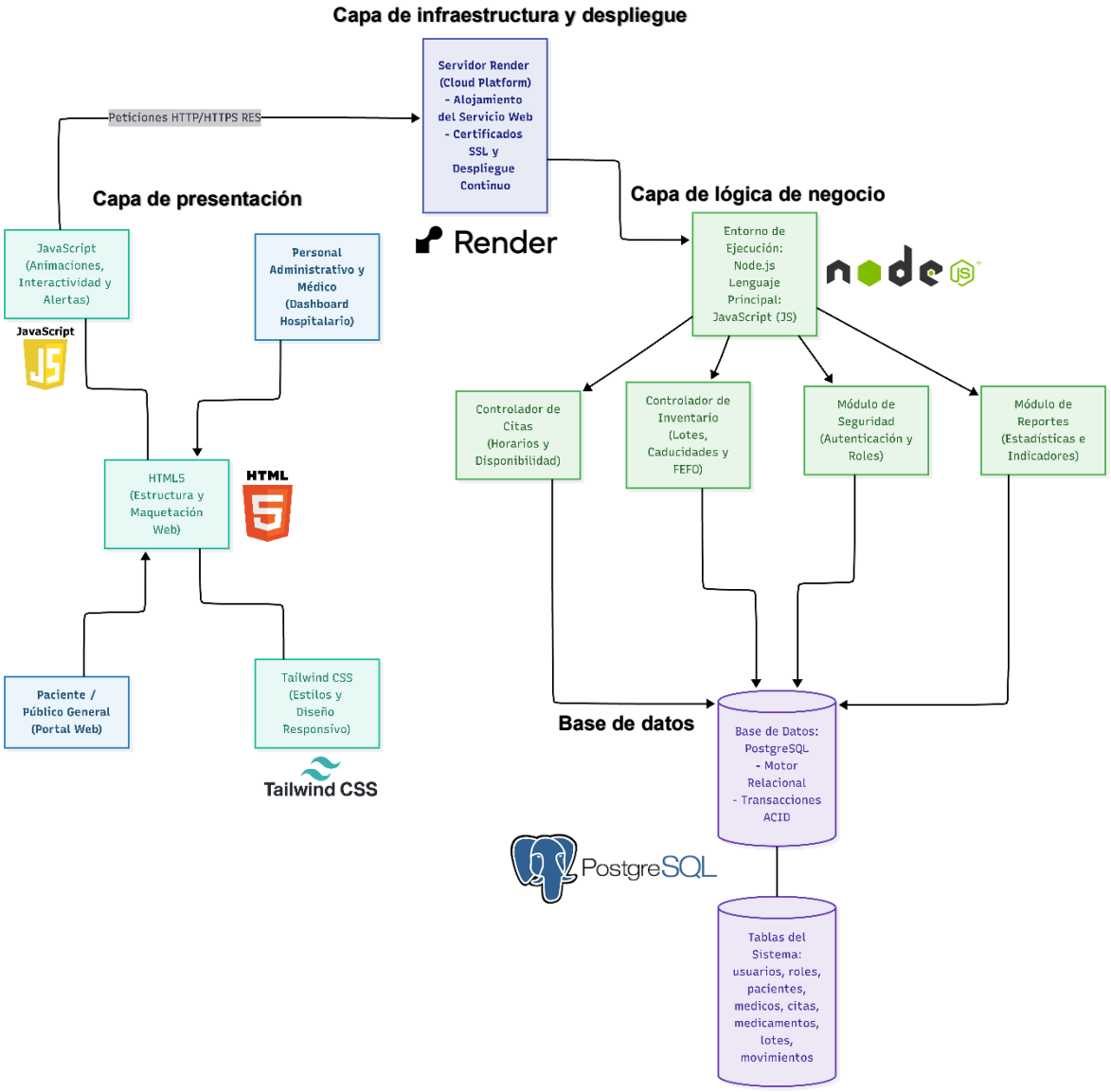


Figura 19 Arquitectura del proyecto

Elaboración propia

3.2 Diagrama entidad relación

Este esquema modela la arquitectura lógica de la base de datos relacional del sistema, representando las conexiones entre los usuarios, roles, pacientes, médicos y la programación de citas, en conjunto con el catálogo de medicamentos, sus lotes por fecha de vencimiento y el historial de transacciones, con el propósito de garantizar la integridad referencial y eliminar inconsistencias en los registros administrativos y clínicos del Hospital Mazahua.



Figura 20 Diagrama entidad relación

Elaboración propia

3.3 Diccionario de datos

Este instrumento técnico describe de manera estructurada las especificaciones de cada tabla y campo del sistema, representando los tipos de datos, longitudes, claves primarias, foráneas y restricciones de obligatoriedad, lo que sirve como norma técnica de referencia para asegurar que la captura, persistencia y consulta de la información de citas e inventario del Hospital Mazahua se realice de forma íntegra, segura y estandarizada.

Tabla: USUARIO

Tabla 2 Tabla de usuario

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_usuario	INT	NO	PK	Identificador único del usuario.
id_rol	INT	NO	FK	Referencia al rol (ROL.id_rol).
nombre	VARCHAR(100)	NO	-	Nombre completo del usuario.
email	VARCHAR(120)	NO	UNICO	Correo electrónico institucional o de acceso.
password	VARCHAR(255)	NO	-	Contraseña del usuario.
estado	BOOLEAN	NO	-	Estado de la cuenta (1: Activo, 0: Inactivo).
fecha_creacion	TIMESTAMP	NO	-	Marca temporal de alta del usuario.

Tabla: PACIENTE

Tabla 3 Tabla de paciente

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_paciente	INT	NO	PK	Identificador único del paciente.
nombre	VARCHAR(60)	NO	-	Nombre(s) de pila del paciente.
apellidos	VARCHAR(80)	NO	-	Apellidos del paciente.
telefono	VARCHAR(15)	SÍ	-	Teléfono móvil o fijo de contacto.
correo	VARCHAR(100)	SÍ	-	Correo electrónico para notificaciones.
direccion	VARCHAR(100)	SI	-	Dirección actual del paciente.
curp	VARCHAR(18)	SÍ	UNICO	Clave Única de Registro de Población.
fecha_nacimiento	DATE	SÍ	-	Fecha de nacimiento del paciente.

Tabla: CITA
Tabla 4 Tabla cita

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_cita	INT	NO	PK	Identificador único de la cita.
id_paciente	INT	NO	FK	Referencia al paciente.
id_medico	INT	NO	FK	Referencia al médico asignado.
fecha_cita	DATE	NO	-	Fecha programada para la consulta.
hora_cita	TIME	NO	-	Hora de inicio programada.
estado_cita	ENUM	NO	-	'Pendiente', 'Confirmada', 'Atendida', 'Cancelada'.
motivo_consulta	VARCHAR(255)	SÍ	-	Descripción breve o síntoma inicial.
fecha_registro	TIMESTAMP	NO	-	Fecha y hora en la que se generó la solicitud.

Tabla: MEDICAMENTO
Tabla 5 Tabla de medicamento

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_medicamento	INT	NO	PK	Identificador único del fármaco.
id_categoria	INT	NO	FK	Clasificación del tipo del medicamento.
codigo_barras	VARCHAR(50)	SÍ	UNICO	Código de identificación estándar.
nombre_comercial	VARCHAR(120)	NO	-	Nombre de marca o presentación comercial.
presentación	VARCHAR(60)	NO	-	Caja, frasco, ampolleta, tabletas.
concentracion	VARCHAR(50)	NO	-	Dosis/concentración (ej. 500 mg, 10 ml).
stock_minimo	INT	NO	-	Valor mínimo para la alerta de desabasto.
stock_actual	INT	NO	-	Cantidad global disponible en farmacia.

Tabla: LOTE_MEDICAMENTO
Tabla 6 Tabla lote de medicamento

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_lote	INT	NO	PK	Identificador único del lote.
id_medicamento	INT	NO	FK	Referencia al medicamento.
numero_lote	VARCHAR(50)	NO	-	Código de lote recibido en farmacia.
fecha_caducidad	DATE	NO	-	Fecha límite de caducidad.
cantidad_inicial	INT	NO	-	Piezas ingresadas originalmente en el lote.
cantidad_disponible	INT	NO	-	Existencia actual de dicho lote.

Tabla: MOVIMIENTO_INVENTARIO
Tabla 7 Tabla movimiento del inventario

Elaboración propia

Atributo	Tipo de Dato	Nulo	Clave	Descripción
id_movimiento	INT	NO	PK	Identificador único de la transacción.
id_usuario	INT	NO	FK	Personal que ejecuta el movimiento (USUARIO.id_usuario).
tipo_movimiento	ENUM	NO	-	'Entrada por compra', 'Entrada por donación', 'Salida por receta', 'Ajuste merma'.
fecha_movimiento	TIMESTAMP	NO	-	Fecha y hora exacta del movimiento.
justificacion	VARCHAR (255)	SÍ	-	Motivo, folio de factura o número de receta médica.

3.4 Diagrama de casos de uso UML

Este diagrama delimita las interacciones entre los usuarios (pacientes, recepcionistas, médicos, farmacéuticos y administradores) y las funciones de la plataforma, representando formalmente qué acciones y permisos específicos tiene cada perfil dentro del Hospital Mazahua para asegurar un acceso controlado a la información.

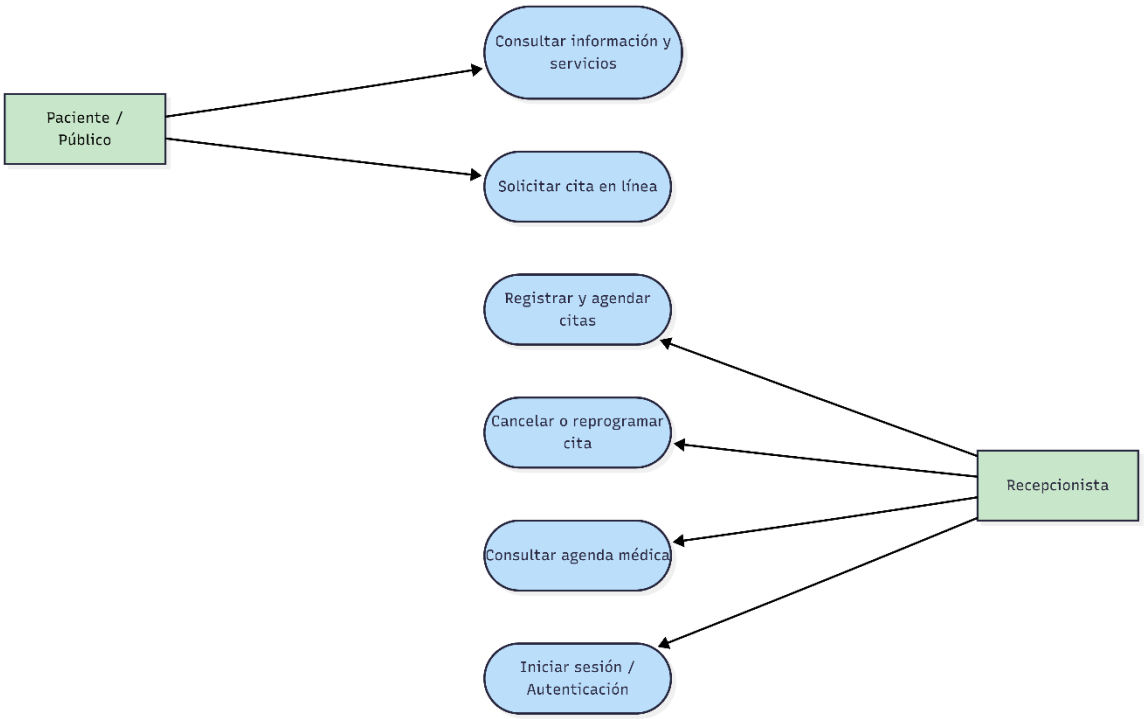


Figura 21 Diagrama de caso de usa, primera vista

Elaboración propia

En la siguiente figura se muestra la continuación del diagrama de caso de uso donde se detallan las operaciones de los demás usuarios como lo son el administrador y el encargado de farmacia.

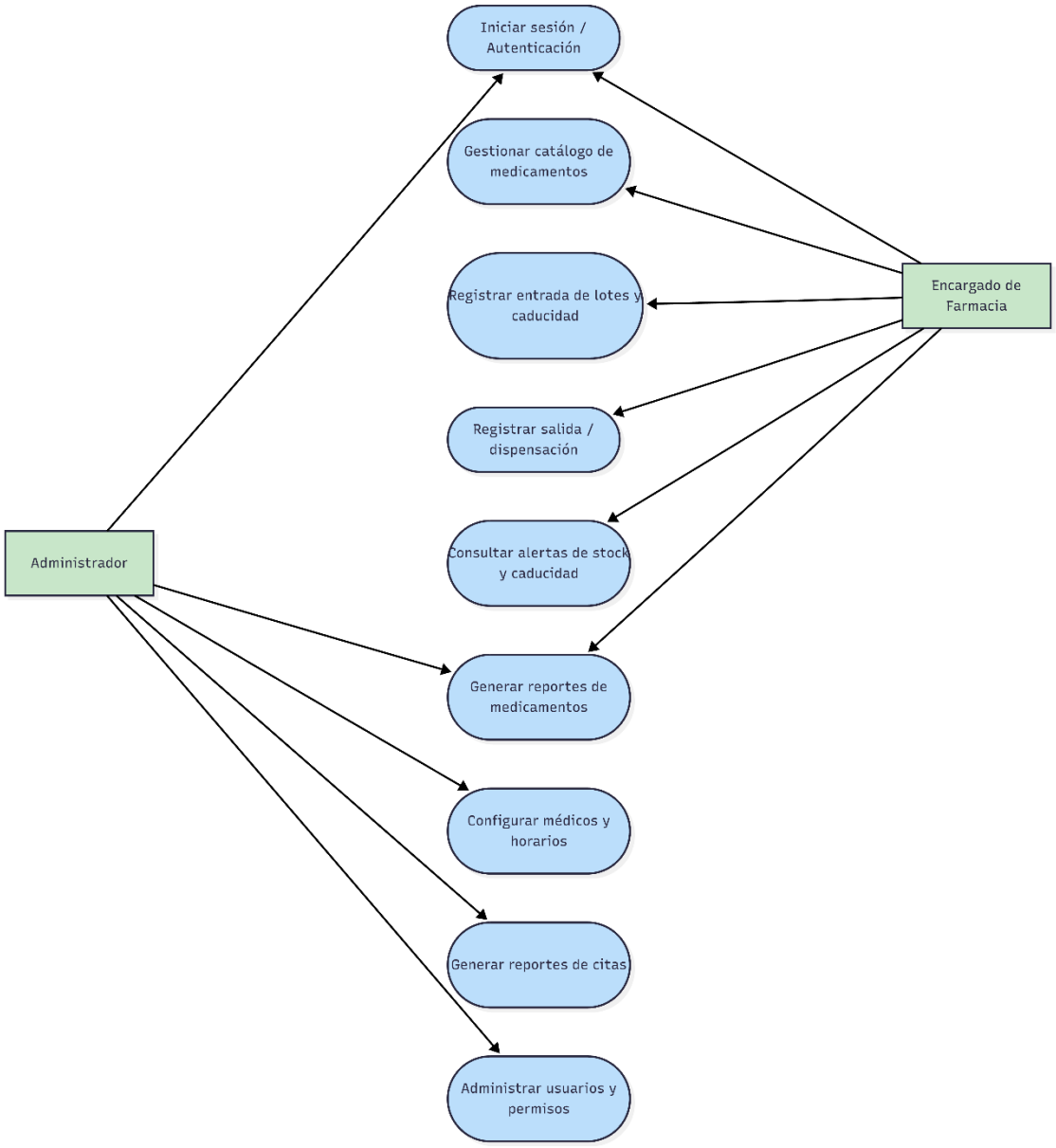


Figura 22 Diagrama de caso de uso, segunda vista

Elaboración propia

3.5 Diagrama de flujo de proceso, ingreso a la plataforma

Este diagrama modela el recorrido de navegación pública del usuario dentro de la plataforma, representando cómo elusuario consulta la cartera de especialidades, ubicación, datos de contacto y valores de la institución, sirviendo como canal de difusión y enlace directo hacia la solicitud de citas médicas.

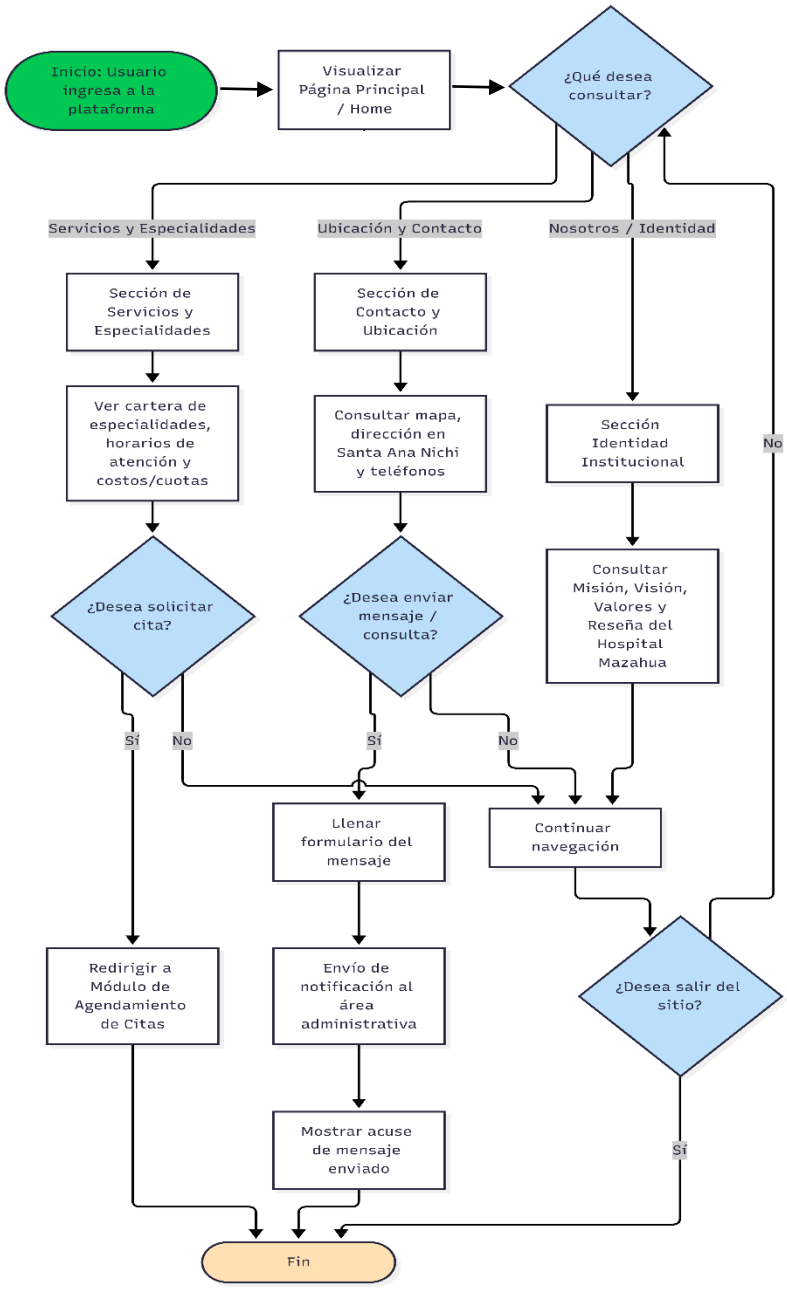


Figura 23 Diagrama de flujo, ingreso a la plataforma

Elaboración propia

3.5.1 Generación de una cita

Este esquema detalla la secuencia lógica para programar una cita tanto en la plataforma, así como en recepción, representando las validaciones paso a paso de disponibilidad médica y prevención de choques en los horarios para garantizar un control ordenado de la agenda hospitalaria.

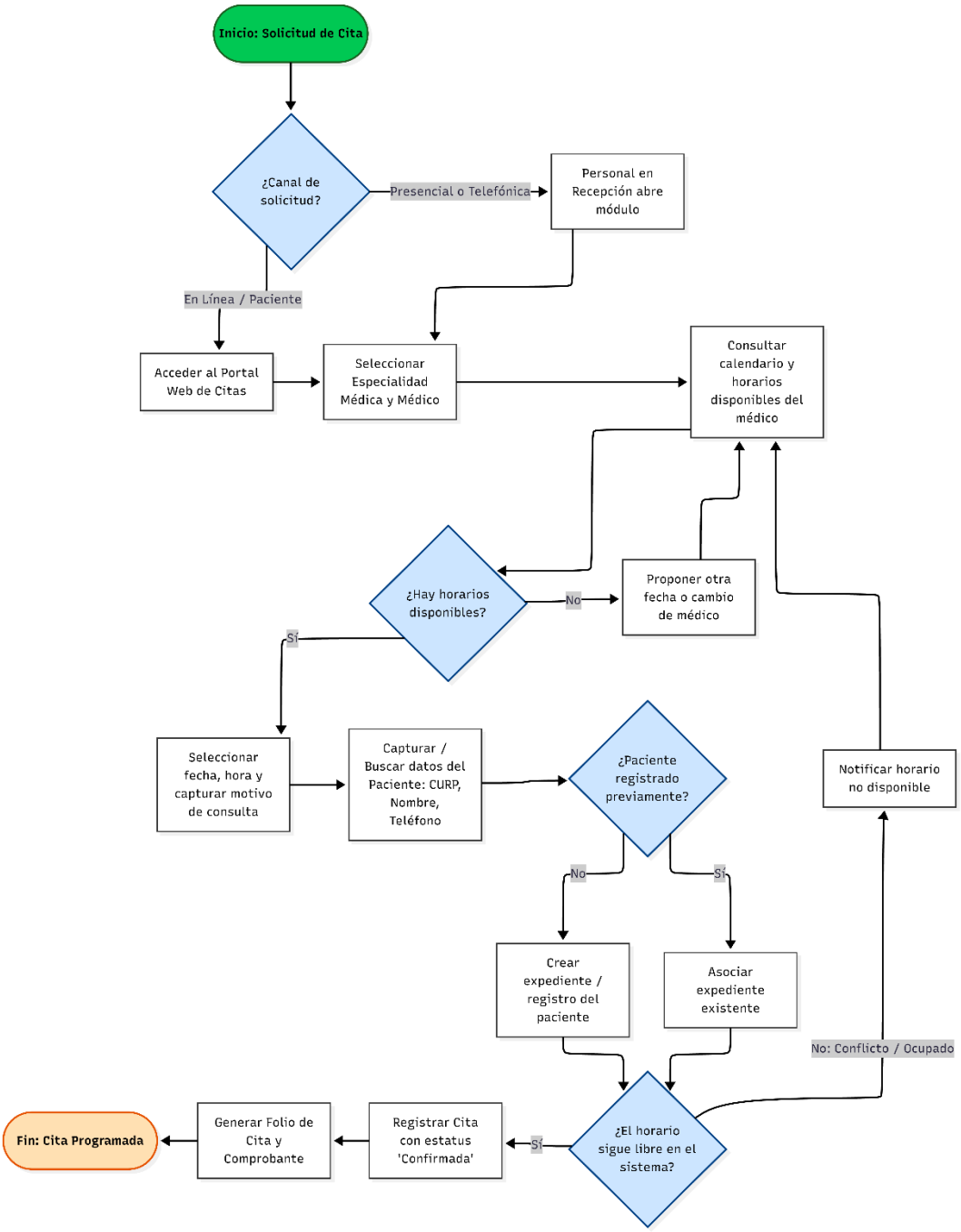


Figura 24 Diagrama de flujo, agendar una cita

Elaboración propia

3.5.2 Administración del inventario

Este diagrama describe las reglas operativas del almacén de farmacia, representando el flujo secuencial de registro de entradas con número de lote y fecha de caducidad, el despacho de medicamentos y el disparo de alertas preventivas ante desabasto o vencimiento.

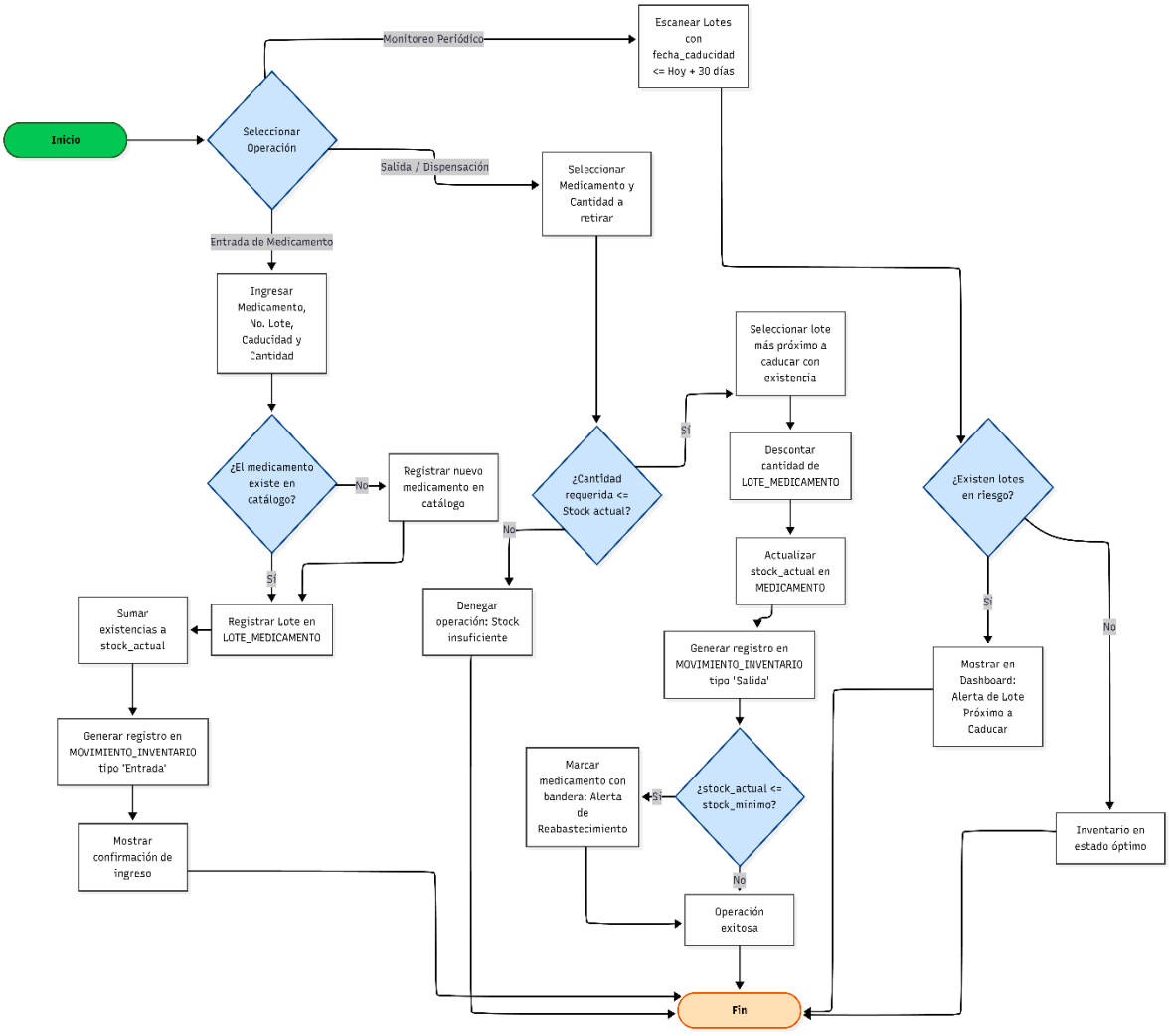


Figura 25 Diagrama de flujo, administración del inventario

Elaboración propia

3.6 Modelado

En los siguientes apartados se describen las pantallas del maquetado desarrollado, el maquetado se realizo en la herramienta de Figma ya que ofrece variedades de opciones para poder desarrollar un buen maquetado.

3.6.1 Maquetado del primer módulo, difusión del hospital.

En la **Figura 26**, el elemento 1 delimita el área de identidad institucional situada en el extremo superior izquierdo, donde se muestran los logotipos oficiales de Hospital Mazahua y COFAS.

El elemento 2 señala el menú de navegación superior en el costado derecho, compuesto por los enlaces directos hacia las secciones informativas de Quiénes somos, Instalaciones y Ubicación y contacto, junto con el icono de acceso de usuario.

El elemento 3 encierra el bloque textual de presentación principal colocado sobre la fotografía panorámica, el cual reúne el nombre legal de la asociación como subtítulo superior, el nombre del centro médico como título de gran jerarquía y un lema breve sobre el compromiso y la vocación de servicio a la comunidad.

El elemento 4 destaca el botón interactivo de llamada a la acción en color rojo con el texto rotulado AGENDAR CITA, diseñado para canalizar de forma prioritaria a los usuarios al sistema de citas.

El elemento 5 corresponde al pie de página ubicado en la base inferior con fondo oscuro, donde se presenta el texto centrado de derechos reservados del año 2026 junto con la razón social oficial de Centros de Orientación Familiar y Salud, I.A.P. (Hospital Mazahua).

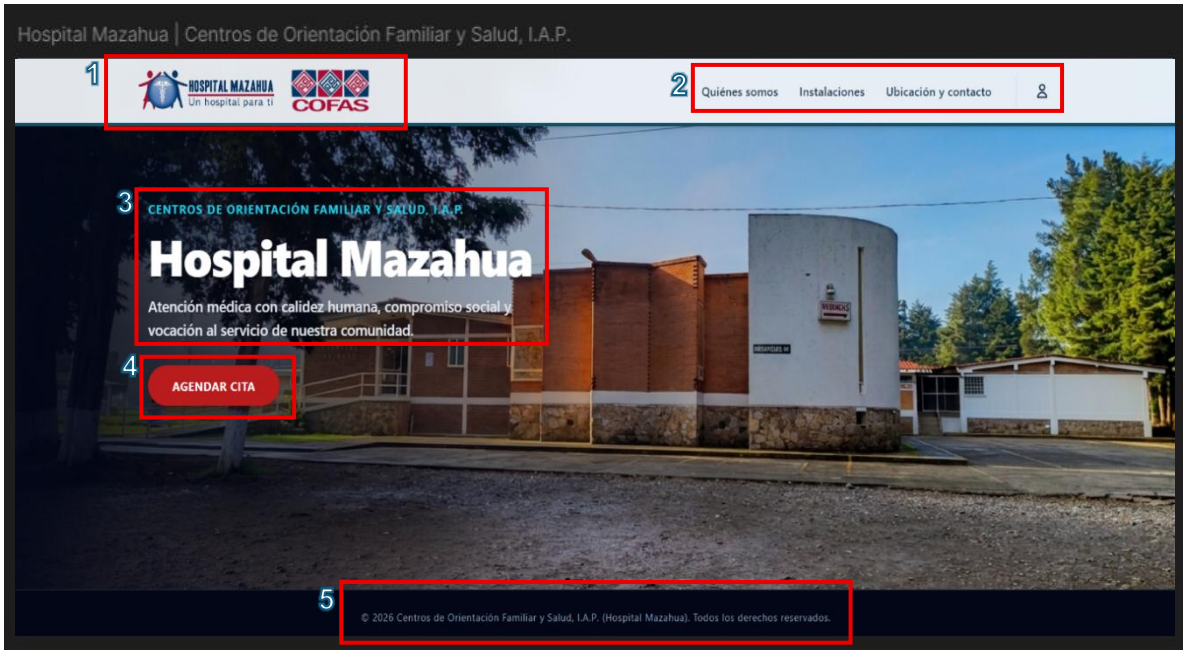


Figura 26 Pantalla de inicio

Elaboración propia

En la **Figura 27**, el elemento 6 señala el botón destacado de llamada a la acción en color rojo con el texto rotulado AGENDAR CITA, ubicado en la parte superior derecha de la barra de navegación para permitir a los usuarios agendar su consulta médica de manera rápida desde esta vista.

El elemento 7 delimita el bloque de filosofía institucional, estructurado horizontalmente en tres tarjetas: la primera detalla la Misión de la organización, la segunda define la Visión a futuro y la tercera, con fondo azul oscuro, desglosa Nuestros Valores en dos columnas de puntos clave.

El elemento 8 encierra el control de navegación interna mediante pestañas que permite alternar la visualización entre Especialidades y Servicios, complementado en su parte inferior por un breve texto descriptivo sobre la atención respaldada por especialistas certificados.

El elemento 9 corresponde a la cuadrícula de presentación de servicios médicos, conformada por tres tarjetas individuales con esquinas redondeadas (Urgencias Médicas, Laboratorio de Análisis Clínicos y Rayos X y Ultrasonido), donde cada una integra una fotografía representativa, el nombre del área y un párrafo con el detalle de las atenciones que brinda.

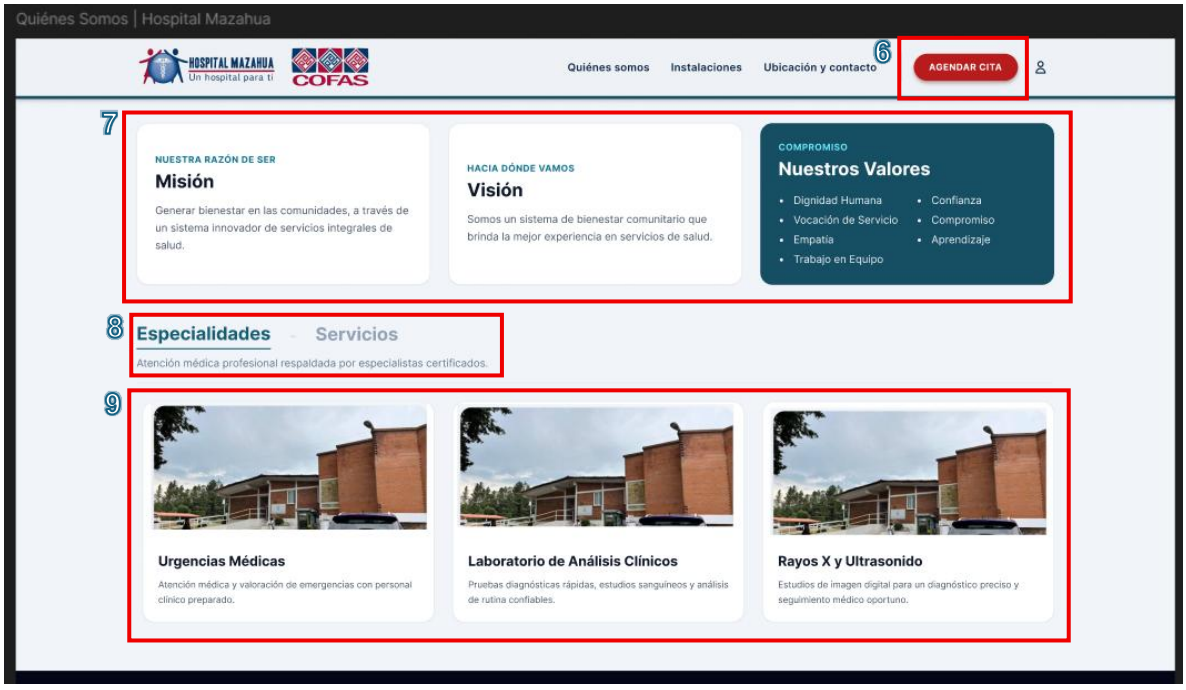


Figura 27 Pantalla de la información general del Hospital Mazahua

Elaboración propia

En la **Figura 28** elemento 10 comprende el bloque de encabezado de la sección, integrado por el título centrado Nuestras Instalaciones con jerarquía visual destacada y un subtítulo breve que describe el confort, la esterilidad y la calidad en la atención médica de los espacios.

El elemento 11 delimita la galería visual organizada en una cuadrícula horizontal de tres tarjetas de bordes redondeados. Cada contenedor presenta una fotografía del centro médico acompañada de su respectivo rótulo de identificación en la esquina inferior izquierda: Fachada Principal, Consultorios Médicos y Recepción y Admisión.

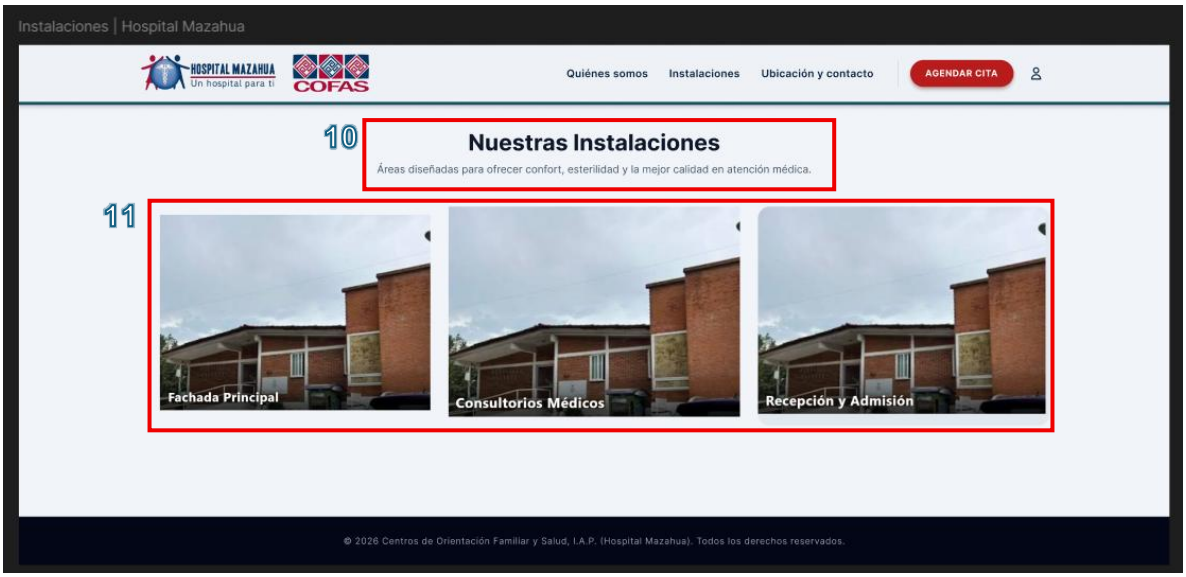


Figura 28 Pantalla de las instalaciones del Hospital Mazahua

Elaboración propia

En la **Figura 29**, el elemento 12 delimita la tarjeta de información situada a la izquierda, la cual incluye el subtítulo de sección, el título Ubicación y horarios, los datos del domicilio oficial, los medios de contacto telefónico y por correo electrónico, un enlace para consultar horarios y un botón destacado en color azul para abrir la ruta en el servicio de mapas que al presionarlo te da la ruta a seguir para poder llegar al Hospital.

El elemento 13 señala el contenedor de la columna derecha que integra el mapa satelital interactivo, el cual ubica geográficamente el centro médico dentro del entorno rural con una ficha de información local en la esquina superior.

El elemento 14 corresponde al bloque centrado de redes sociales ubicado debajo del área principal, identificado con el texto SÍGUENOS EN: junto a los enlaces directos hacia los perfiles institucionales de Facebook e Instagram.

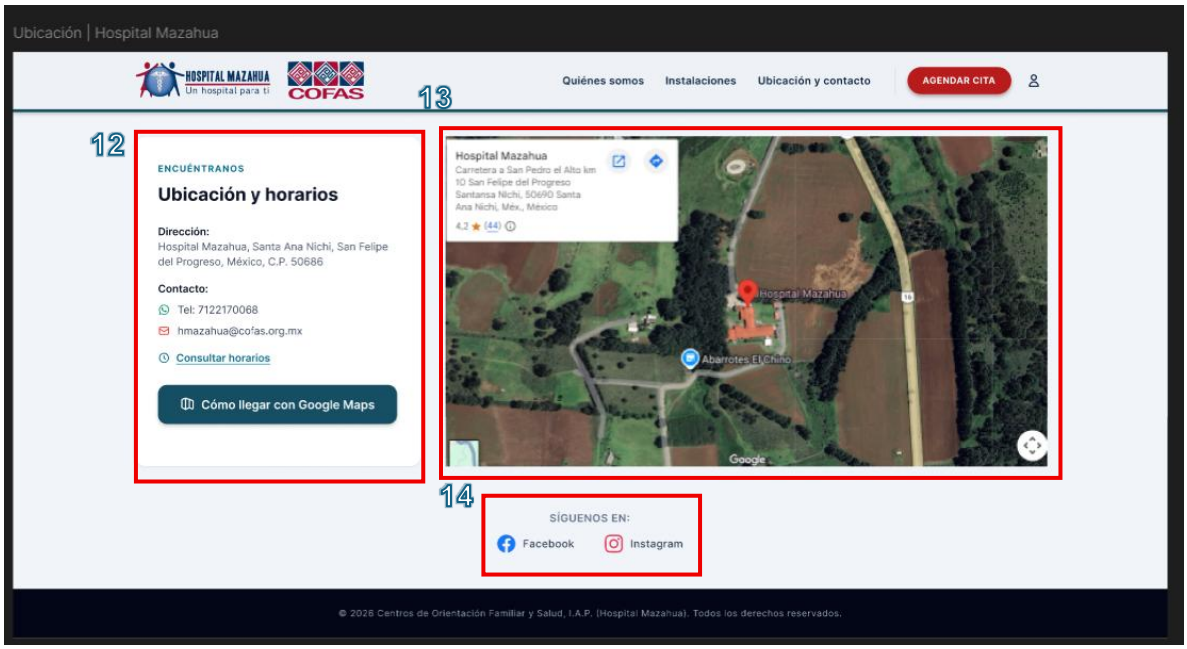


Figura 29 Pantalla donde se consulta la ubicación y el contacto del Hospital Mazahua

Elaboración propia

El elemento 15 delimita la ventana emergente centrada para el acceso al sistema sobre un fondo oscurecido y desenfocado, integrando en su esquina superior derecha un botón de cierre con cruz, en la parte superior central un icono de usuario seguido por el encabezado Iniciar sesión y el subtítulo Ingresa con tus credenciales, el campo de captura para Usuario con icono de arroba.

En el campo de Contraseña con icono de candado cuyos caracteres permanecen ocultos al inicio mediante puntos medios pero que cuenta con un botón interactivo en su extremo derecho para visualizar u ocultar la clave escrita, el enlace inferior derecho para recuperar la cuenta mediante Olvidé mi contraseña, y el botón principal de confirmación en color azul marino rotulado Ingresar, el cual valida los datos introducidos impidiendo el acceso a la plataforma si las credenciales no son correctas.

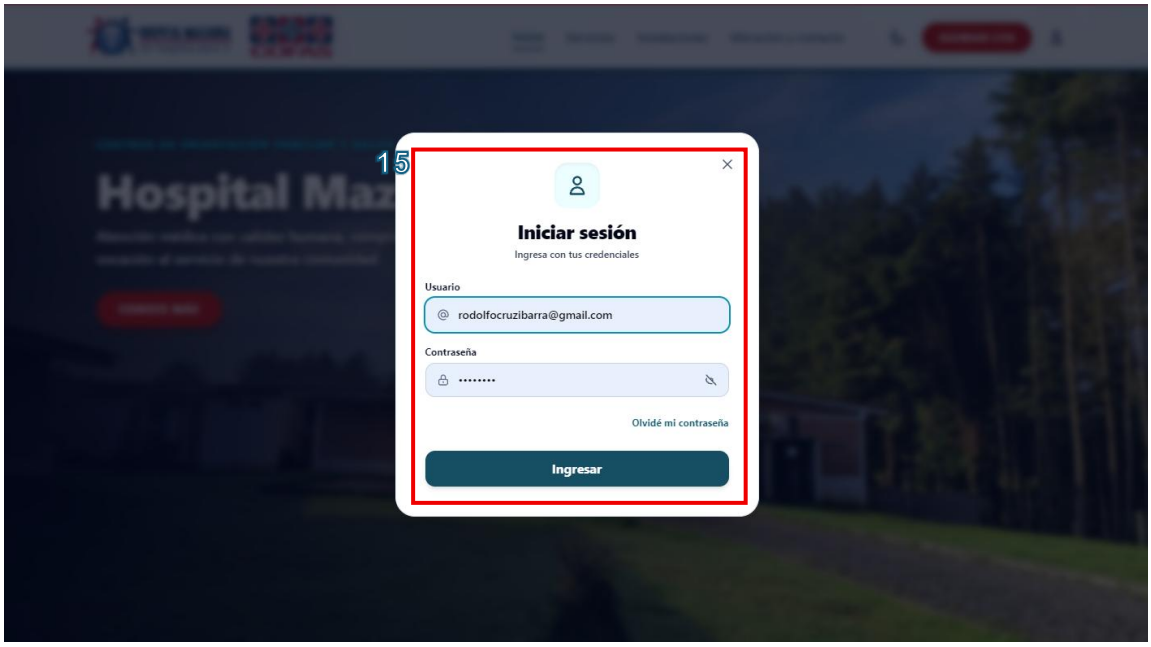


Figura 30 Pantalla de inicio de sesión

Elaboración propia

CAPÍTULO IV. DESARROLLO

Se documenta todo el proceso que se tuvo que llevar a cabo para el desarrollo del proyecto, como por ejemplo la instalación de extensiones de preparación, la instalación del entrono virtual, estructura de las carpetas para alojar los archivos y de igual manera la conexión hacia la base de datos.

4.1. Instalación de software.

Antes de inicializar repositorios o dependencias locales, se requiere contar con los siguientes entornos instalados:

4.1.1 Node.js (LTS):

Se descarga el instalador desde su página oficial, eligiendo el sistema operativo, en este caso es para Windows.

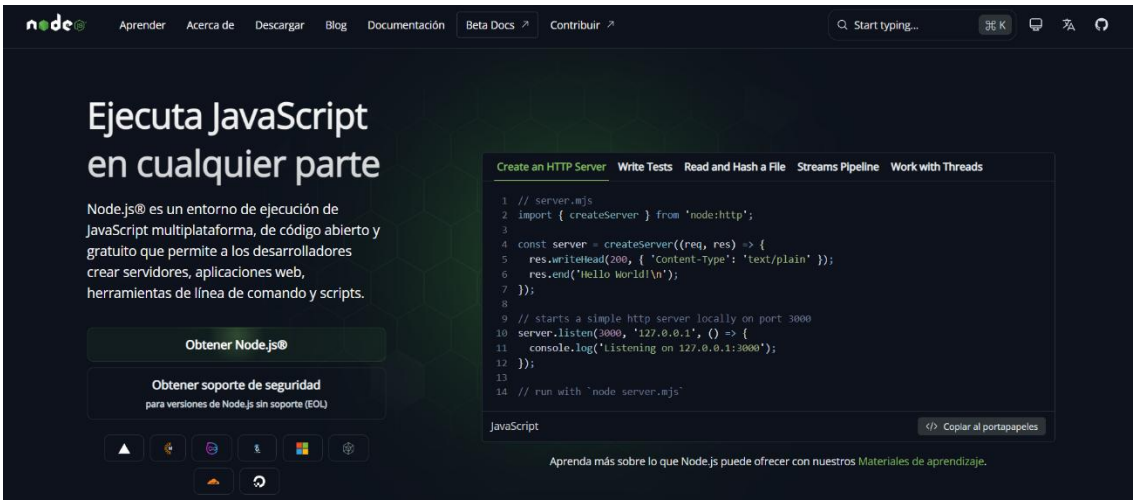


Figura 31 Instalador de Node.js

Captura de pantalla, recuperado de: https://nodejs.org/es

4.2 PostgreSQL: Se descarga el instalador en su página oficial para poder instalarlo en el sistema operativo Windows.

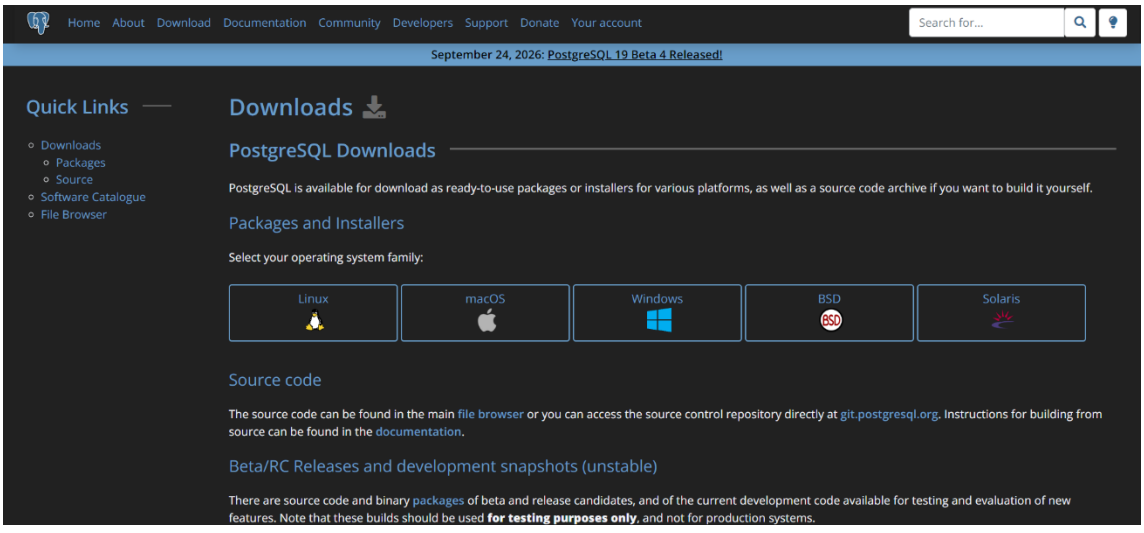


Figura 32 Página oficial de PostgreSQL

Captura de pantalla, recuperado de: https://www.postgresql.org/download/

4.2.1 Programa pgAdmin4:

Al ejecutar el instalador de PostgreSQL se debe de instalar el programa de pgAdmin4 para poder crear la base de datos.

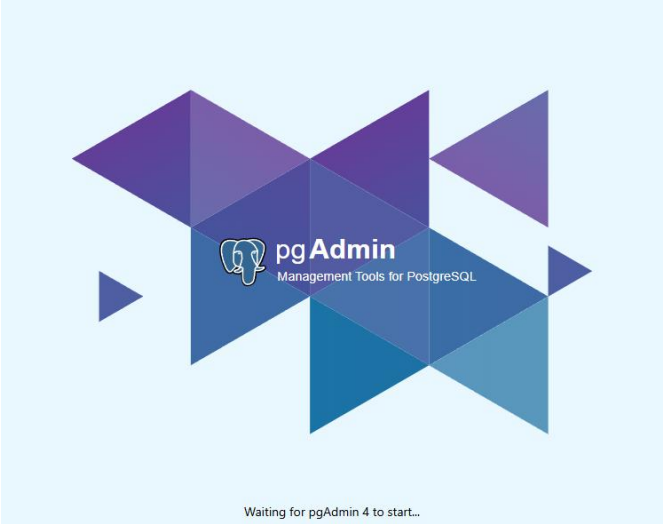


Figura 33 Programa pgAdmin4

Captura de pantalla

4.3. Instalación de Visual Studio Code.

Para poder a iniciar a desarrollar la estructura del proyecto se necesita instalar el editor de código Visual Studio Code, ya que este editor de código ofrece grandes beneficios para desarrollar un buen proyecto.

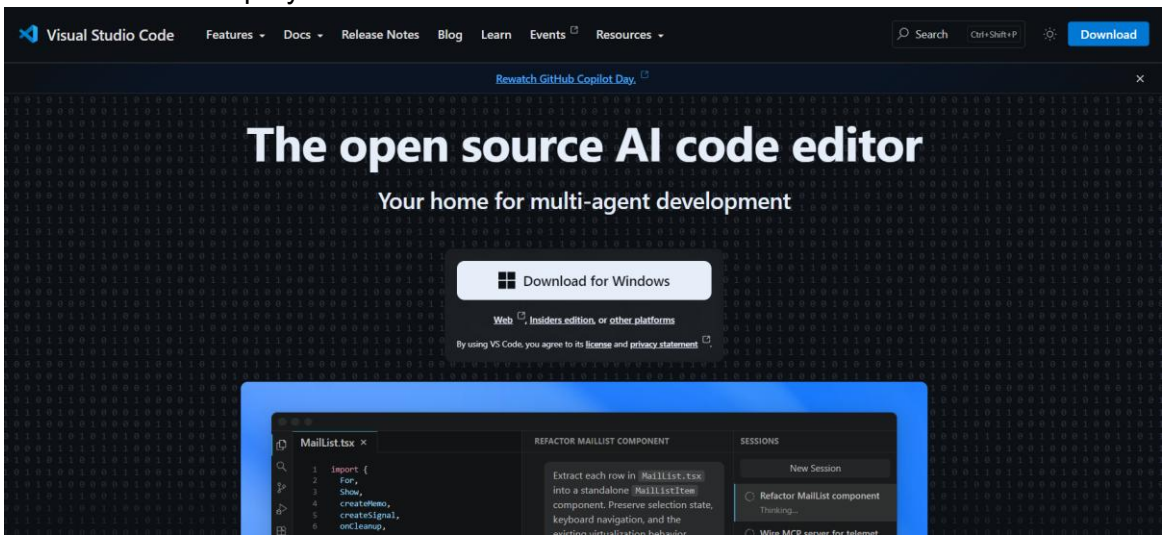


Figura 34 Instalador de Visual Studio Code

Captura de pantalla, obtenido de: https://code.visualstudio.com/

4.4. Estructura del proyecto.

El proyecto mantiene una estructura ordenada para separar lógica de servidor, fuentes visuales y distribución compilada para una mayor distribución de todos los archivos para que se pueda encontrar cada uno de ellos.

En la **Figura 34** se muestra la estructura actual del proyecto que se esta desarrollando con el entorno de desarrollo establecido.

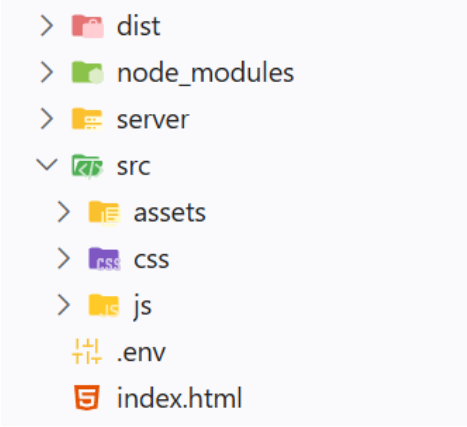


Figura 35 Estructura del proyecto

Elaboración propia

4.5. Inicialización del entorno Node.js y dependencias.

Para poder instalar todas las dependencias dentro del proyecto se ejecutan en terminal yh dejar que termine el proceso de instalación.

4.5.1. Creación del archivo de configuración del proyecto:

```
npm init -y
```

4.5.2 Instalación de dependencias de producción backend y servicios:

```
npm install express cors pg dotenv nodemailer
```

- **express:** Framework minimalista para crear la API REST y gestionar rutas.
- **cors:** Middleware para permitir peticiones entre diferentes orígenes en Live Server y Node.js.
- **pg:** Driver oficial para conectar Node.js con PostgreSQL mediante consultas SQL.
- **dotenv:** Carga segura de variables de entorno desde el archivo .env.
- **nodemailer:** Motor de envío de correos electrónicos vía SMTP.

4.5.3. Instalación de dependencias de desarrollo (Tailwind CSS):

npm install -D tailwindcss

4.6. Configuración y Compilación de Tailwind CSS

Para poder instalar Tailwind CSS, en el proyecto dentro de Visual Studio Code se ejecutan en una terminal y dejar que termine el proceso.

4.6.1. Inicializar el archivo de configuración:

npx tailwindcss init

4.7. Creación y edición de tailwind.config.js: Estando en la raíz del proyecto se debe de crear un archivo de configuración para tailwind.

En **la Figura 35** se muestra el fragmento de código que funciona para.

tailwind.config.js > ...

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./*.html",
    "./src/**/*.js"
  ],
  darkMode: 'class',
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Figura 36 Código de tailwind.config.js

Elaboración propia

4.8. Creación del archivo base input.css con las directivas:

En la raíz del proyecto con la siguiente dirección se crea el archivo input.css (src/css/input.css).

En la Figura 36 se detalla el código de configuración que funciona para

src > css > input.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

```

Figura 37 Código de configuración de CSS

Elaboración propia

4.9. Creación de la Base de Datos.

Para la creación de la base de datos se tiene que realizar en el programa pgAdmin 4 que se instaló previamente.

Dentro del programa, en su terminal de SQL se crea primero la base de datos nombrado h_mazahua.

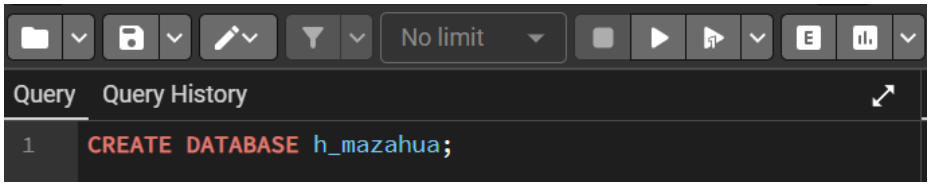


Figura 38 Creación de la base de datos

Elaboración propia

Una vez creada la base de datos ya se pueden ir creando las diferentes tablas que conformaran a la base de datos, como primero se tiene la tabla de usuarios, en la **Figura 38** se muestra el código para la creación de la primera tabla con sus atributos correspondientes. Para la creación de las diferentes tablas es el mismo proceso, solo cambian los datos.

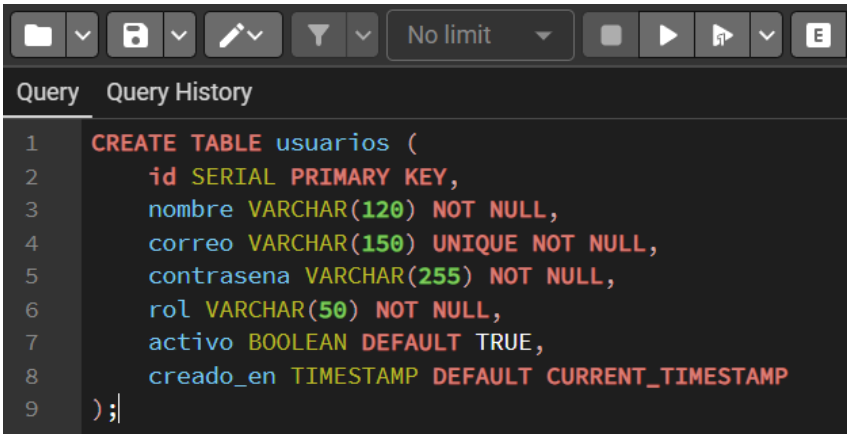


Figura 39 Código SQL para la creación de la tabla usuarios

Elaboración propia

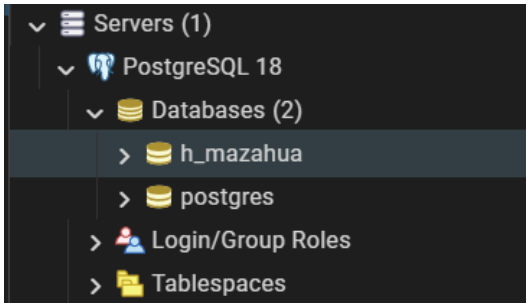


Figura 40 Verificación de la creación de la BD

Elaboración propia

4.10. Configuración de variables de entorno (.env).

En la raíz del proyecto se crea el archivo **.env** que son las variables de entorno, que son básicamente las credenciales que se necesitan antes de realizar la conexión a la base de datos.

.env

```
PORT=3000
DB_USER=postgres
DB_HOST=localhost
DB_DATABASE=h_mazahua
DB_PASSWORD=admin123
DB_PORT=5432
```

Figura 41 Credenciales del archivo .env

Elaboración propia

4.11. Conexión de la base de datos en Node.js.

En este paso se debe de crear un archivo en la carpeta de servar, ahí se creara el archivo server.js que tiene como función realizar la conexión del proyecto hacía la base de datos de PostgreSQL utilizando las credencias de .env.

server > db.js > ...

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_DATABASE,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});

pool.on('connect', () => {
  console.log('Conexión exitosa a la base de datos h_mazahua en PostgreSQL');
});

pool.on('error', (err) => {
  console.error('Error inesperado en el cliente PostgreSQL:', err);
  process.exit(-1);
});

module.exports = pool;
```

Figura 42 Código para la conexión hacia la BD

Elaboración propia

4.11.1. Comando para la activación del servidor.

Con el siguiente comando se usa solamente para que el proyecto en general se pueda conectar a la base de datos de PostgreSQL de forma local.

node server/server.js

CONCLUSIONES

Hasta la fecha, el desarrollo de la Plataforma Integral de Gestión Hospitalaria para el Centro de Orientación Familiar y Salud, I.A.P. (Hospital Mazahua) ha alcanzado un avance significativo del 35% al consolidar su arquitectura base cliente-servidor se ha establecido la infraestructura de base de datos relacional mediante PostgreSQL, integrando un pool de conexiones optimizado que interactúa directamente con una API REST construida sobre Node.js y Express. Sobre esta estructura backend se implementó el control de acceso inicial, el cual valida credenciales en tiempo real, verifica el estado activo de las cuentas de usuario y gestiona el inicio de sesión seguro dentro del entorno del sistema.

Asimismo, se concluyó íntegramente el módulo de recuperación de contraseña, integrando un flujo de verificación en dos pasos mediante la generación de códigos dinámicos de seis dígitos enviados por correo electrónico a través de servicios SMTP de Nodemailer. Este mecanismo incluye la validación previa de la existencia del usuario en la base de datos, un cronómetro de caducidad estricto para la confirmación del código enviado y una interfaz adaptativa con soporte para visibilidad de contraseñas.
Con la incorporación de Tailwind CSS para la maquetación visual a estructurado las interfaces dinámicas, intuitivas y responsivas, asegurando una experiencia de usuario fluida tanto en computadoras de escritorio como en dispositivos móviles dentro del hospital. La separación clara entre la capa de presentación y la lógica de servidor no solo incrementa la seguridad en el manejo de datos sensibles, sino que también facilita el mantenimiento del código fuente y prepara el terreno para las pruebas de integración en las siguientes fases del proyecto.

Con este avance se garantiza seguro y escalable, dejando la plataforma completamente preparada para proseguir con el desarrollo de los módulos de control de inventario de medicamentos, seguimiento de lotes con alertas de caducidad y la programación de citas médicas.

REFERENCIAS

- Awan, A. A. (2024, 24 de abril). Descubre SQLAlchemy: un tutorial para principiantes con ejemplos. DataCamp. https://www.datacamp.com/es/tutorial/sqlalchemy-tutorial-examples
- Celesti, A., Fazio, M., y Villari, M. (2019). Un estudio sobre bases de datos NoSQL orientadas a columnas para el análisis de macrodatos en entornos de nube. IEEE Transactions on Cloud Computing, 8(4), 1105–1117. https://doi.org/10.1109/TCC.2019.2905781
- Comité de Familiares de Asistencia Social. (s. f.). Hospital Mazahua. Recuperado el 18 de septiembre de 2026, de https://cofasorg.wordpress.com/hospitales/hospital-mazahua-2/
- Consorcio Mexicano de Hospitales. (s. f.). Hospital Mazahua. Recuperado el 18 de septiembre de 2026, de https://cmhac.com/directory/hospital-mazahua/
- Coronel, C., y Morris, S. (2019). Sistemas de bases de datos: Diseño, implementación y administración (13.ª ed.). Cengage Learning.
- Databricks. (2026, 14 de enero). ¿Qué es una base de datos PostgreSQL? Databricks. https://www.databricks.com/es/blog/what-is-postgresql-database
- Derakhshan, R., Stantic, B., y Sattar, A. (2019). Procesamiento y gestión de datos de series temporales: Arquitecturas, desafíos y direcciones futuras. ACM Computing Surveys, 52(3), 1–36. https://doi.org/10.1145/3313788
- Elmasri, R., y Navathe, S. B. (2017). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson Educación.
- Equipo de Contenidos de GoDaddy. (2025, 27 de marzo). Python: Qué es, para qué sirve y por qué aprenderlo. GoDaddy Resources - LATAM. https://www.godaddy.com/resources/latam/desarrollo/python-que-es
- Google Cloud. (s. f.). ¿Qué es una plataforma de aplicaciones? Google Cloud. Recuperado el 18 de septiembre de 2026, de https://cloud.google.com/discover/what-is-an-application-platform?hl=es
- Huet, P. (2022, 21 de enero). Qué es Tailwind CSS y por qué deberías usarlo. OpenWebinars. https://openwebinars.net/blog/que-es-tailwind-css-y-por-que-deberias-usarlo/

- MDN Web Docs. (2023). Validación de formularios de datos. Mozilla Corporation. https://developer.mozilla.org/es/docs/Learn/Forms/Form_validation
- MyTaskPanel Consulting. (2023, 7 de noviembre). ¿Qué es Flask? Beneficios y usos. CEEI Valencia - EmprenemJunts. https://ceeivalencia.emprenemjunts.es/?op=8&n=30392
- OWASP. (2021). OWASP Top 10 - 2021: Los diez riesgos más críticos de seguridad en aplicaciones web. Open Web Application Security Project. https://owasp.org/Top10/
- OWASP. (2023). Hoja de referencia de validación de entradas [Input validation cheat sheet]. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html.
- Pressman, R. S., y Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill Interamericana.
- Robinson, I., Webber, J., y Eifrem, E. (2015). Bases de datos de grafos: Nuevas oportunidades para datos conectados (2.ª ed.). O'Reilly Media.
- Robert, G. (2026, 19 de junio). What is CSS3? BrowserStack. https://www.browserstack.com/guide/what-is-css3
- Sadalage, P. J., y Fowler, M. (2013). NoSQL esencial: Una breve guía del mundo emergente de la persistencia políglota. Addison-Wesley.
- Sadashiv, N., y Kumar, S. M. (2011). Computación en clúster, malla y nube: Una comparación detallada. En Actas de la 6.ª Conferencia Internacional sobre Informática y Educación (ICCSE 2011) (pp. 477–482). IEEE. https://doi.org/10.1109/ICCSE.2011.6028683
- Saavedra, J. A. (2023, 1 de junio). ¿Qué es HTML5 y para qué sirve? Ebac. https://ebac.mx/blog/que-es-html5
- Saavedra, J. A. (2023, 4 de junio). Qué es Github y para qué sirve: una guía para principiantes. Ebac. https://ebac.mx/blog/que-es-github
- Saavedra, J. A. (2023, 12 de septiembre). ¿Qué es JavaScript y cómo funciona? Ebac. https://ebac.mx/blog/que-es-javascript
- Silberschatz, A., Korth, H. F., y Sudarshan, S. (2020). Fundamentos de bases de datos (7.ª ed.). McGraw-Hill Interamericana.
- Sommerville, I. (2016). Ingeniería del software (10.ª ed.). Pearson Educación.

ANEXOS

Cronograma de actividades

CRONOGRAMA ACTIVIDADES

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

Unidad de Estudios Superiores San José del Rincón

"Plataforma Integral para la Gestión de Medicamentos y Generación de Citas del Centro de Orientación Familiar y Salud, I. .

Responsable: Rodolfo Cruz Ibarra	**CRONOGRAMA DE ACTIVIDADES**
Objetivo: Desarrollar una plataforma integral para la gestión de medicamentos y generación de citas del Centro de Orientación Familiar y Salud, I. A. P. (Hospital Mazahua), mediante la digitalización y centralización de la información, con el fin de optimizar los procesos administrativos y mejorar la atención al paciente.	forma Integral para la Gestión de Medicamentos y Generación de Citas del Centro de Orientación Familiar y Salud, I. A. P. (Hospital M

ACTIVIDAD	PERIODO: 01 – septiembre 2026 al 20 enero 2026																					
	MES	SEPTIEMBRE				OCTUBRE				NOVIEMBRE				DICIEMBRE				ENERO			
	Semanas	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Planeación del proyecto	P																				
	R																				
Actividad #1. Análisis de requerimientos del sistema, mediante entrevistas con el personal del hospital para definir funciones y alcances.	P																				
	R																				
Actividad #2. Diseño de la base de datos, estableciendo las tablas y relaciones necesarias para el manejo de medicamentos, citas y usuarios.	P																				
	R																				
Actividad #3. Diseño de interfaces de usuario, elaborando los prototipos de interfaz de las pantallas principales del sistema.	P																				
	R																				
Actividad #4. Configuración del entorno de desarrollo y definición de la arquitectura del sistema.	P																				
	R																				
Actividad #5. Desarrollo del módulo para la difusión de la información del hospital.	P																				
	R																				
Actividad #6. Desarrollo del módulo de gestión de inventario de medicamentos, incluyendo el registro de entradas, salidas y existencias.	P																				
	R																				
Actividad #7. Desarrollo del módulo de agendamiento de citas médicas, con la disponibilidad de médicos y horarios.	P																				
	R																				
Actividad #8. Desarrollo del módulo de reportes e indicadores, para el consumo de medicamentos y el estado de las citas.	P																				
	R																				
Actividad #9. Desarrollo del apartado de difusión institucional, con la información general del hospital.	P																				
	R																				
Actividad #10. Integración de los módulos desarrollados, verificando su correcta comunicación dentro de la plataforma.	P																				
	R																				
Actividad #11. Elaboración del plan y casos de prueba, definiendo los escenarios a validar en cada módulo.	P																				
	R																				
Actividad #12. Primera revisión y corrección de errores detectados en el sistema.	P																				
	R																				
Actividad #13. Optimización y ajustes finales del sistema, con base en los resultados de las pruebas.	P																				
	R																				
Actividad #14. Implementación del sistema en el entorno del centro de salud.	P																				
	R																				
Actividad #15. Capacitación al personal administrativo sobre el uso de la plataforma.	P																				
	R																				
Actividad #16. Puesta en marcha y seguimiento inicial del funcionamiento del sistema.	P																				
	R																				
Actividad #17. Segunda revisión y corrección ante posibles fallos del sistema una vez que se ponga en marcha.	P																				
	R																				

COMENTARIOS GENERALES :	**Responsable: Rodolfo Cruz Ibarra**	**Alumno**
	Criterios Pro. Reali. Pendi. No Re	**Rodolfo Cruz Ibarra**

Requerimientos

1. Requerimientos Funcionales:

Son las funciones, procesos y servicios directos que el sistema debe realizar para satisfacer las necesidades operativas de la institución.

Módulo 1: Informativo y difusión institucional.

- Consulta de Servicios e Información: La plataforma debe permitir al público en general consultar la cartera de servicios médicos, especialidades disponibles, horarios de atención, cuotas de recuperación, misión, visión y ubicación geográfica del hospital.
- Contacto y orientación: El sistema debe ofrecer un formulario de contacto web para que los usuarios envíen solicitudes de información o dudas directamente al área administrativa.

Módulo 2: Gestión y agendamiento de citas

- Solicitud de cita en línea: El sistema debe permitir a los pacientes solicitar citas médicas seleccionando especialidad, médico, fecha y horario disponible sin necesidad de acudir previamente al hospital.
- Registro de citas presenciales o telefónicas: El personal de recepción debe poder registrar, agendar y confirmar citas médicas para pacientes que asisten en ventanilla o llaman por teléfono.
- Validación de disponibilidad y choque de horarios: El sistema debe verificar en tiempo real que el horario y médico seleccionados no cuenten con una cita previa asignada, impidiendo la duplicidad o cruce de horarios.
- Gestión del estatus de citas: El sistema debe permitir actualizar el estado de cada cita programada a: Pendiente, Confirmada, Atendida o Cancelada.
- Registro y consulta de pacientes: El sistema debe permitir registrar y consultar expedientes básicos de pacientes mediante su CURP o datos personales para evitar registros duplicados.

Módulo 3: Inventario y control de medicamentos

- Catálogo de medicamentos: El sistema debe permitir registrar, editar y clasificar fármacos por nombre comercial, principio activo, presentación, concentración y categoría terapéutica.
- Registro de salidas: El sistema debe registrar las salidas de medicamentos descontando automáticamente el stock y aplicando la regla de priorizar el lote con fecha de caducidad más próxima (criterio FEFO).
- Alertas de caducidad y stock mínimo: La plataforma debe generar notificaciones visuales automáticas cuando un medicamento esté por debajo de su umbral mínimo de existencias o cuando un lote esté próximo a vencer.

Módulo de Reportes y Administración

- Reportes de consumo de medicamentos: El sistema debe generar reportes filtrados por periodo (diario, semanal, mensual) sobre entradas, salidas, mermas y existencias actuales de fármacos para la toma de decisiones administrativas.
- Reportes de indicadores de citas: La plataforma debe emitir estadísticas sobre el volumen de citas agendadas, atendidas, ausencias y cancelaciones por médico o especialidad.
- Exportación de información: El sistema debe permitir exportar los reportes de inventario y citas a formatos estándar como PDF y hojas de cálculo (Excel/CSV).
- Control de acceso y roles de usuario: El sistema debe implementar autenticación segura (login) y restringir el acceso a los módulos según el rol asignado: Administrador, Recepcionista, Médico o Encargado de Farmacia.

2. Requerimientos No Funcionales

Son las cualidades de calidad, rendimiento, restricciones técnicas y de seguridad que debe cumplir el software para operar de manera óptima.

Seguridad

- Protección de credenciales): Las contraseñas de los usuarios deben almacenarse de forma cifrada mediante algoritmos seguros de derivación de claves.
- Control de sesiones y autorización): El sistema debe cerrar automáticamente la sesión tras un periodo prolongado de inactividad e impedir el acceso a rutas privadas mediante tokens o validación de sesiones activas.

Usabilidad y diseño

- Diseño adaptable: La interfaz del sitio debe ser responsiva y adaptarse correctamente a diferentes dispositivos y resoluciones de pantalla (computadoras de escritorio, laptops, tablets y teléfonos móviles).
- Facilidad de uso: Las pantallas y formularios deben contar con mensajes claros de validación de campos, alertas visuales intuitivas (colores de advertencia para caducidades) y requerir una curva de aprendizaje mínima para el personal hospitalario.

Mantenibilidad y Compatibilidad

- Compatibilidad con navegadores: La plataforma debe ser totalmente funcional en los navegadores web modernos y actualizados (Google Chrome, Mozilla Firefox, Microsoft Edge y Safari).